

# SUDP: Secret-Use Delegation Protocol for Agentic Systems

**Xiaohang Yu**

Imperial College London  
Science Intelligence  
x.yu21@imperial.ac.uk

**Hejia Geng**

University of Oxford  
Science Intelligence  
hejia@scienceintelligence.org

**Xinmeng Zeng**

Stanford University  
xinzeng@stanford.edu

**William Knottenbelt\***

Imperial College London  
w.knottenbelt@imperial.ac.uk

## Abstract

Agentic systems increasingly act with user secrets for APIs, messaging platforms, and cloud services. Today’s agent runtimes typically implement *authorization by exposure*: enabling action often means placing a reusable secret, or a reusable artifact derived from it, inside the runtime, so a transient prompt-injection or tool-side compromise becomes durable account compromise. Existing defenses cover adjacent pieces such as secret storage, scoped delegation, sender-constrained tokens, and runtime monitoring, but leave the combined agentic obligation without a common specification: an untrusted autonomous requester should be able to cause a user-authorized secret-backed operation without gaining reusable authority over it. We formalize this as the *Agent Secret Use* (ASU) problem and identify seven security properties any solution must satisfy, spanning authorization integrity and secret confidentiality. We propose the *Secret-Use Delegation Protocol* (SUDP), in which a requester proposes a canonical operation, the user authorizes it with a fresh authenticator-backed grant, and a custodian redeems the grant to perform the bounded use; reusable authority never crosses the requester boundary. We specialize SUDP for LLM-driven agents, where it applies whenever a tool call would exercise user-enrolled authority-bearing material. Under standard cryptographic assumptions, SUDP satisfies all seven properties when integrated with a hardware-rooted runtime. A reference implementation is available at <https://github.com/xhyumiracle/sudp>.

## 1 Introduction

A coding agent holding a GitHub token can be steered by an injected README into pushing code to a public repository: once a reusable secret is inside the agent runtime, indirect prompt injection becomes durable account compromise [Greshake et al., 2023, OWASP Foundation, 2024]. Tool-augmented language-model agents now routinely perform such authority-bearing actions: they call model and platform APIs, send messages, and authenticate to third-party tools [Schick et al., 2023, Mialon et al., 2023, Yao et al., 2023, OpenAI, 2023, Anthropic, 2024a], backed by API keys, OAuth tokens, or signing keys held in environment variables, databases, or secret managers [HashiCorp, 2024a, Amazon Web Services, 2024] and handed to the agent for ad-hoc use. Across successive agent-safety benchmarks [Zhan et al., 2024, Zhang et al., 2024, Debenedetti et al., 2024, Zhou et al.,

\*Corresponding author.

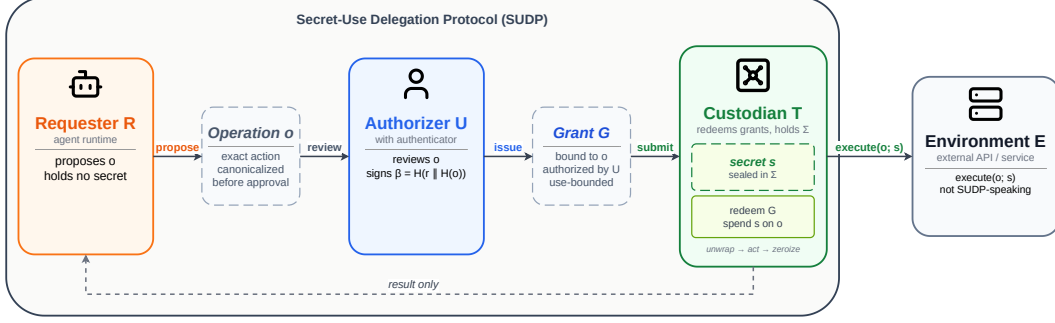


Figure 1: Schematic of SUDP. Three protocol roles (Requester  $R$ , Authorizer  $U$ , Custodian  $T$ ) together with the environment  $E$  (outside the protocol). The operation  $o$  and the grant  $G$  are the protocol’s authorization artifacts:  $R$  proposes  $o$ ;  $U$  reviews  $o$  and issues  $G$ , whose signature binds  $H(o)$ , freshness  $r$ , and acting credential identifier  $cid_{c^*}$ ;  $T$  redeems  $G$  and uses the secret  $s$  sealed in state  $\Sigma$  to execute  $o$  at  $E$ . Only the result returns to  $R$ , and reusable authority never crosses the requester boundary. Protocol flow, key hierarchy, and authorization-artifact structure are detailed in §5.

2025, Qiao et al., 2025], agent runtimes have continued to fall to prompt injection and tool misuse on secret-adjacent tasks despite improving base models; under the bearer-secret pattern, every such failure becomes durable authority compromise.

The question for agentic systems is: *how can an agent be authorized to cause a secret-backed operation without ever gaining reusable authority over it?* Today’s model bundles capability and possession into a single artifact, a pattern we call *authorization by exposure*. Existing approaches address neighboring dimensions: capability-based authority [Dennis and Van Horn, 1966, Miller, 2006], scoped delegation [Hardt, 2012, Birgisson et al., 2014], sender-constrained bearers [Fett et al., 2023], user-mediated authenticators with derivable material [Hodges et al., 2021, FIDO Alliance, 2025], and standardized cryptographic plumbing [Krawczyk and Eronen, 2010, Dworkin, 2012, Schaad and Housley, 2002, Housley and Dworkin, 2009, Barnes et al., 2022, Rescorla, 2018]; none provides a common specification for *authorization without giving the requester reusable authority*. We compose these elements into a unified protocol whose unit of delegation is one authorized operation, not the secret itself, and frame this as an *agent capability* problem: the bottleneck on what AI agents can autonomously do is not only reasoning ability but the safety of the authorization layer they operate against.

**Contributions.** (i) **Agent Secret Use as a problem class** (§3). We give a vocabulary-fragmented field a common lens: a single problem statement and an outcome-based security-property taxonomy under which claims for secret-mediation, delegation-token, and TEE-backed systems become checkable judgments against named properties.

(ii) **The SUDP protocol** (§5). SUDP makes secret safety a cryptographic invariant rather than deployment discipline: every Phase III execution is covered by a fresh authorizer-signed grant cryptographically bound to a specific canonical operation, and reusable authority never crosses the requester boundary. Anti-substitution, operation binding, and replay resistance hold by construction.

(iii) **Agentic specialization and analysis** (§6–9). The agentic specialization draws the requester boundary to cover the LLM-context exposure surface, and separates the LLM-API and tool-call layers of agent secret use so that SUDP applies exactly where user-enrolled authority-bearing material is at stake. Even a fully prompt-injected agent can at most propose operations subject to explicit authorization, never gain reusable authority. We instantiate SUDP with standard cryptographic components, introducing no new cryptographic assumption, and prove it under one main theorem with five supporting propositions covering authorization integrity, secret confidentiality, recipient-protected export, and both wrapping-epoch and authority-level forward secrecy.

## 2 Related Work

**Agent systems and external capabilities.** AutoGPT and AutoGen made long-horizon, tool-using agents a concrete software pattern rather than only a prompting style [Significant Gravitas, 2023, Wu et al., 2023]. Recent products push the same pattern toward browser, desktop, and multi-step task execution: Manus, OpenAI Operator, and Anthropic’s computer-use interface [Manus Team, 2025, OpenAI, 2025b, Anthropic, 2024b].

**Secret management, authorization, and delegation.** Capability-based access control [Dennis and Van Horn, 1966, Miller, 2006], OAuth 2.0 bearers [Hardt, 2012, Jones and Hardt, 2012], DPoP [Fett et al., 2023], OAuth Rich Authorization Requests [Lodderstedt et al., 2023], GNAP [Richer, 2024], MCP authorization [Anthropic and MCP Working Group, 2025], and Macaroons [Birgisson et al., 2014] address scoped delegation but pass possession-bearing or reusable artifacts through the requester. WebAuthn [Hodges et al., 2021] and its PRF extension [FIDO Alliance, 2025] provide phishing-resistant user authentication and authenticator-bound key derivation, primitives that SUDP builds on (§5). Secret managers and KMS services [HashiCorp, 2024a, Amazon Web Services, 2024] provide mature storage, access control, and rotation, but their privileged backend assumes the authorization decision is upstream of the requester; SUDP complements them at the authorization layer. A recent line of agentic delegation work extends web-identity and provenance constructs to agent principals: Authenticated Delegation [South et al., 2025] binds agent credentials to a verified human delegator; SAGA [Syros et al., 2025] provider-mediate inter-agent communication; Agentic JWT [Goswami, 2025] binds each agent action to a workflow-step assertion; HDP [Dalugoda, 2026] targets cross-hop provenance; and OIDC-A [Nagabhushanaradhya, 2025] extends OIDC with agent identity. Each touches a different subset of the ASU axes; the per-axis comparison is in §4.2.

**Runtime defenses, monitors, and benchmarks.** A large body of work hardens the agent or contains the consequences of misbehavior: instruction-data separation and prompt-engineering defenses [Chen et al., 2024a,b, Hines et al., 2024], capability-tracking interpreters [Debenedetti et al., 2025], per-tool policy engines [Shi et al., 2025, Wang et al., 2025, Tsai and Bagdasarian, 2025, Liu et al., 2026], verify-before-commit mitigations [Lin et al., 2026], guardrail stacks [Chennabasappa et al., 2025], and execution sandboxes [Ruan et al., 2023, Wu et al., 2025, Zhou et al., 2025]. These approaches reason about what the agent does or contain where it does it; SUDP constrains what a secret can be used for and composes with them rather than substituting. Recent adaptive evaluations show that stronger attacks bypass published behavioral defenses [Nasr et al., 2025], motivating protocol-level guarantees beyond behavioral filtering, and a large-scale empirical study of LLM agent skills finds widespread credential-leakage exposure in deployed code [Chen et al., 2026]. Agent-safety benchmarks across model generations [Zhan et al., 2024, Debenedetti et al., 2024, Zhang et al., 2025, Zhou et al., 2025, Vijayvargiya et al., 2026, Qiao et al., 2025, El Yagoubi et al., 2026] consistently report nontrivial attack success on prompt injection, tool misuse, and cross-tool privacy leakage; systematizations [Kim et al., 2026, Deng et al., 2026, Li et al., 2026, Ying et al., 2026] identify capability-constrained secret use at the Execution stage as an open gap in the layered defense stack.

## 3 Agent Secret Use

*Agent Secret Use* (ASU) arises when an autonomous requester must cause a user-authorized operation without ever gaining reusable authority over that operation.

### 3.1 The Problem

Agent Secret Use poses a conflict between three parties. A user  $U$  holds authority at an external environment  $E$ , exercised through authority-bearing material  $s$  (Definition 1) that  $E$  accepts on  $U$ ’s behalf for operations in a space  $\mathcal{O}_E$ . An autonomous requester  $R$ , acting on  $U$ ’s behalf, lies outside  $U$ ’s trust boundary:  $R$  must never gain reusable authority (Definition 2) over those operations. The unmediated native interface therefore forces a bad choice (Figure 2). Definition 3 states the problem; §3.2 fixes the adversary, and §3.3 formalizes the security properties a mechanism must enforce to solve it.

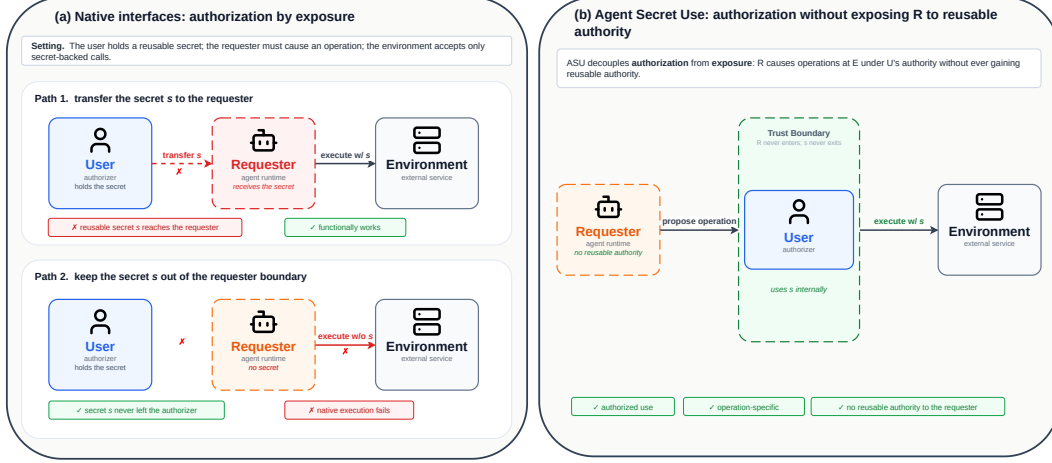


Figure 2: **Agent Secret Use as a structural problem.** (a) Native secret-backed interfaces implement *authorization by exposure*: handing  $s$  to  $R$  functionally works but gives  $R$  reusable authority over operations at  $E$ ; keeping  $s$  outside  $R$  preserves confidentiality but the native call is rejected by  $E$ . (b) ASU decouples authorization from exposure:  $R$  proposes the operation,  $U$  authorizes it, and  $s$  stays inside a trust boundary outside  $R$ , so  $R$  never gains reusable authority over operations at  $E$ .

**Definition 1 (Authority-bearing material).** *Material  $s$  is authority-bearing at  $E$  if possession of  $s$ , or of material derived from  $s$  that  $E$  accepts, enables operations at  $E$  under the authority of the party that enrolled  $s$ .*

**Definition 2 (Reusable authority).** *Reusable authority over operations at  $E$  is the capability to cause more than one invocation of operations at  $E$ , whether via direct possession of authority-bearing material (Definition 1) or via any derived artifact  $E$  accepts.*

**Definition 3 (Agent Secret Use Problem).** *Agent Secret Use is the problem of enabling an untrusted autonomous requester  $R$ , acting on  $U$ 's behalf, to cause operations in  $\mathcal{O}_E$  to be executed at  $E$ , without ever exposing  $R$  to reusable authority (Definition 2) over what  $U$  explicitly authorizes.*

### 3.2 Adversary Model

We parameterise the adversary  $\mathcal{A}$  by capability:

- $\mathcal{A}^R$ : controls  $R$ .
- $\mathcal{A}^{\text{net}}$ : controls the network (observes, modifies, drops, replays, and reorders any message).
- $\mathcal{A}^{\text{state}}$ : reads at rest the persistent state of any component a solution introduces to hold authority-bearing material.
- $\mathcal{A}^{\text{mem}}$ : reads the runtime memory of such a component during an in-progress operation.

$\mathcal{A}$  may also query  $U$  on any  $o \in \mathcal{O}_E$ ;  $U$  returns either a decision attributable to  $U$  or a refusal, per a policy the model leaves abstract.

The ASU problem (Definition 3) is stated against the baseline  $(\mathcal{A}^R, \mathcal{A}^{\text{net}})$ ;  $\mathcal{A}^{\text{state}}$  and  $\mathcal{A}^{\text{mem}}$  are robustness extensions targeted by the corresponding axes of §3.3.

**Goal.**  $\mathcal{A}$  wins by either *extraction*, obtaining authority-bearing material (Definition 1) and thereby reusable authority (Definition 2) over operations at  $E$ , or *misuse*, causing  $E$  to execute some  $o$  that  $U$  did not explicitly authorize. §3.3 formalizes the security properties that rule these out.

### 3.3 Security Properties

We formalize the Agent Secret Use problem (Definition 3) via seven security properties, grouped along the classical confidentiality / integrity distinction [McCumber, 1991, Saltzer and Schroeder, 1975]. The four *core* properties (AV, OB, UB, CRC) together preclude the two adversarial wins of

§3.2: AV, OB, and UB rule out *misuse*; CRC rules out *extraction*. We say a mechanism *solves the ASU problem* if it enforces these four core properties under the baseline adversary ( $\mathcal{A}^R, \mathcal{A}^{\text{net}}$ ). The three *robustness extensions* (CSB, CMC, RFS) target the stronger adversaries  $\mathcal{A}^{\text{state}}$  and  $\mathcal{A}^{\text{mem}}$ ; they are not required for the baseline solution, and a mechanism that holds secret material only within  $U$  satisfies them trivially.

#### Authorization integrity.

- **AV: Authorization Verifiability.** Any successful execution induced through the mechanism is covered by an authorization event attributable to  $U$  under the mechanism’s stated identity and trust assumptions. An authenticator-bound assertion  $U$  produces for a specific operation provides stronger direct evidence than an identity provider’s assertion about  $U$ .
- **OB: Operation Binding.** An authorization is bound to a specific operation  $o$ ; the mechanism does not induce execution of any operation semantically distinct from  $o$ .
- **UB: Use Boundedness.** Execution induced through the mechanism cannot exceed the bounds  $U$  authorized.

#### Secret confidentiality.

- **CRC: Confidentiality under Requester Compromise.** Compromise of the requester does not expose authority-bearing material (Definition 1).
- **CSB: Confidentiality under Storage Breach.** Compromise of the persistent state of a secret-holding component does not expose the secret. Encryption whose unlock key resides in the same compromised state does not satisfy CSB.
- **CMC: Confidentiality under Runtime-Memory Compromise.** Compromise of the runtime memory of a secret-processing component does not expose the secret. Hardware-rooted TEEs may satisfy CMC within their threat model [Costan and Devadas, 2016]; software-only schemes relying on OS process isolation generally do not.
- **RFS: Rotation Forward Secrecy.** Compromise of secret material in one rotation epoch does not expose material in any other epoch. Rotating only the wrapping key while reusing the same authority-bearing secret is epoch-key isolation, not authority-level forward secrecy.

Deployment concerns (client or authenticator compromise, logs and telemetry, IPC, crash dumps, supply-chain code injection, side channels, out-of-band copying) lie outside the seven security axes.

## 4 Positioning

We position SUDP along two layers. §4.1 locates the ASU problem among other problem classes that place the authority-holding boundary differently, and assesses where existing scheme families fall short of the ASU boundary. §4.2 then evaluates SUDP against representative deployed systems along the security-property axes of §3.3.

### 4.1 Relation to Prior Paradigms

Agent Secret Use isolates a specific agentic collision: an autonomous requester that initiates operations on the user’s behalf must stay outside the boundary that holds authority-bearing material for those operations. This is a confused-deputy setting [Hardy, 1988] with an added confidentiality constraint: the deputy must not gain reusable authority it could be induced to misuse. Prior paradigms answer neighboring versions of secret-use delegation by placing the authority-holding boundary at different points. Agent Secret Use does not prescribe a location; it fixes a negative boundary condition:  $R$  must stay outside any boundary that contains authority-bearing material accepted at  $E$ . Placement is left to the realization.

#### Prior paradigms, by boundary placement.

- *Bearer-token authorization* (OAuth 2.0 bearer [Hardt, 2012, Jones and Hardt, 2012], DPoP [Fett et al., 2023]) places the boundary at the token itself: possession or proof-of-possession material is passed to the requester, which remains trusted with reusable authority.

Table 1: Positioning along the seven security axes of §3.3 plus FI. Symbol semantics and row notes follow the table. Ratings reflect strict ASU definitions against public documentation at the time of writing; they are not a general security ranking of these systems.

|                                                              | Secret Confidentiality |     |     |     | Authorization Integrity |    |    | Deploy. |
|--------------------------------------------------------------|------------------------|-----|-----|-----|-------------------------|----|----|---------|
|                                                              | CRC                    | CSB | CMC | RFS | AV                      | OB | UB | FI      |
| Bearer-token baseline                                        | ×                      | ×   | ×   | ×   | ×                       | ×  | ×  | ✓       |
| HashiCorp Vault AI-agent identity pattern [HashiCorp, 2024b] | ×                      | △   | ×   | ×   | △                       | ×  | ×  | ✓       |
| Aegis [Warren, 2025]                                         | ✓                      | △   | ×   | ×   | ×                       | ×  | ×  | ✓       |
| Claude Agent SDK proxy [Anthropic, 2026]                     | △                      | ×   | ×   | ×   | ×                       | ×  | ×  | ✓       |
| IronClaw [NEAR AI, 2026]                                     | ✓                      | ✓   | △   | ×   | ×                       | ×  | ×  | ×       |
| Authenticated Delegation [South et al., 2025]                | ×                      | ×   | ×   | ×   | △                       | ×  | ×  | ✓       |
| SAGA [Syros et al., 2025]                                    | ×                      | ×   | ×   | ×   | ✓                       | △  | ×  | △       |
| OIDC-A [Nagabhushanaradhya, 2025]                            | ×                      | ×   | ×   | ×   | △                       | ×  | ×  | ✓       |
| HDP [Dalugoda, 2026]                                         | ×                      | ×   | ×   | ×   | ✓                       | ×  | △  | ✓       |
| Agentic JWT [Goswami, 2025]                                  | ×                      | ×   | ×   | ×   | △                       | △  | △  | ✓       |
| <b>SUDP (this paper)</b>                                     | ✓                      | ✓   | △   | ✓   | ✓                       | ✓  | ✓  | ✓       |

- *Authorization-server delegation* (OAuth Rich Authorization Requests [Lodderstedt et al., 2023], G NAP [Richer, 2024], MCP authorization [Anthropic and MCP Working Group, 2025]) places the boundary at a centralized authorization server that mints fine-grained tokens, but the token still lands at the requester.
- *Centralized secret storage* (Vault, KMS services [HashiCorp, 2024a, Amazon Web Services, 2024]) places the boundary at the storage system and addresses access control and rotation, but the authorization decision sits upstream of the requester rather than being bound to specific operations.
- *Hardware-rooted boundaries* (HSMs, secure enclaves, TEE-backed runtimes [Costan and Devadas, 2016, Sabt et al., 2015]) place the boundary inside tamper-resistant hardware that confines signing keys or code; this addresses host-memory confidentiality but does not by itself authorize specific agentic operations.
- *Transaction authentication ceremonies* (closest analogue), exemplified by PSD2 dynamic linking, banking 2FA, Secure Payment Confirmation [W3C Web Payments Working Group, 2026], and WebAuthn UV [Hodges et al., 2021] ceremonies, place the boundary at a trusted user-facing client cooperating with a relying party. This is the prior paradigm closest to ASU’s operation-bound user approval, but it assumes the user interacts directly with a trusted client rather than through an autonomous agentic requester.

A separate body of concurrent work targets the same agentic setting as SUDP: *agentic delegation protocols* (Authenticated Delegation [South et al., 2025], SAGA [Syros et al., 2025], OIDC-A [Nagabhushanaradhya, 2025], HDP [Dalugoda, 2026], Agentic JWT [Goswami, 2025]). These are not prior paradigms but contemporary peers; we compare them per-axis in §4.2.

## 4.2 Comparison Along the Security Axes

We apply the taxonomy of §3.3 to position SUDP among representative deployed systems for agent secret use.

**Framework Independence (FI).** Alongside the seven security axes we add a deployability axis: a defense is *FI-positive* if its specification is adoptable by any LLM-agent runtime without a specific agent SDK, planner, tool scheduler, or TEE-backed execution architecture. FI is kept separate from the security axes because it is an adoption property, not a security outcome.

**What the table evaluates.** Table 1 restricts attention to systems that make secret-confidentiality or authorization claims. Behavioral monitors [Chennabasappa et al., 2025, Luo et al., 2025], policy DSLs [Shi et al., 2025, Wang et al., 2025, Tsai and Bagdasarian, 2025], taint-tracking interpreters [Debenedetti et al., 2025], and execution sandboxes [Wu et al., 2025] operate at orthogonal layers (prompt / tool-call / data-flow / cross-app isolation) and are not captured by our axes; they compose with SUDP rather than compete.

**Rating criteria.** Axis abbreviations follow §3.3. Each system is evaluated under its theoretical strongest configuration documented in public specifications or supported deployment modes, applied uniformly across rows. ✓ marks an axis natively satisfied in that configuration; △ marks an axis the documentation claims but the system does not natively deliver (typically because the property holds only via composition with an external substrate, or only in a deployment mode the specification does not commit to); × marks an axis the documentation does not establish under our definition. A × means “does not satisfy this property under our definition,” not “the system lacks security value”; many cited systems are well-designed answers to neighboring problems that place their trust boundaries elsewhere.

*Bearer-token baseline* characterizes the prevalent agent-framework pattern (LangChain [LangChain, 2024], AutoGen [Wu et al., 2023], CrewAI [CrewAI, 2024], OpenAI Agents SDK [OpenAI, 2025a]) rather than ranking specific frameworks. *IronClaw* and SUDP both rate △ on CMC because the property holds only when the custodian runs inside a hardware-rooted runtime. SUDP’s *RFS* at the authority level additionally depends on the deployment honouring §5.8’s rotation/revocation cadence at *E*.

**Landscape.** The table shows two neighboring clusters. *Secret managers and credential proxies* (Vault, Aegis, Claude Agent SDK proxy, IronClaw) keep reusable authority outside the requester and cover Secret Confidentiality to varying degrees, but issue no operation-bound *U*-rooted authorization. *Agentic delegation protocols* (Authenticated Delegation, SAGA, OIDC-A, HDP, Agentic JWT) build authorization, scoping, and provenance on OAuth/OIDC infrastructure but leave reusable authority inside the requester boundary. TEE-backed deployments (represented by IronClaw) form a composition layer rather than a third cluster: they close host-memory confidentiality at the cost of a specific execution substrate, and SUDP composes with them to close CMC.

**SUDP’s position.** SUDP natively satisfies AV, OB, UB, CRC, CSB, and RFS, and is FI-positive; CMC is closed via composition with a hardware-rooted runtime. Guarantees outside SUDP’s scope (code-integrity attestation, multi-hop delegation provenance, behavioral-policy correctness, taint tracking, sandbox isolation) compose with rather than substitute for the protocol: a deployment may run SUDP inside a TEE (closing CMC), behind a runtime monitor or policy DSL, alongside taint tracking or sandboxed tool execution, or chain SUDP’s single-hop authorization into a multi-hop delegation protocol.

## 5 Secret-Use Delegation Protocol

SUDP is a protocol-level answer to capability-constrained secret use. The unit of delegation is the *use* of a secret for one specific authorized operation *o*, not the secret itself: the requester is delegated the right to *cause* an authorized use, the custodian is delegated the right to *perform* the use within *U*’s signed bounds, and reusable authority never crosses the requester boundary. This section first fixes SUDP’s roles and trust model (§5.1), then defines SUDP as an abstract protocol over generic cryptographic interfaces (§5.2–5.7) together with the rotation discipline that delivers per-credential forward secrecy (§5.8). The agentic-system interface induced by the protocol is given in §6; the standards-based cryptographic instantiation in §7; architectural design choices in §8.

### 5.1 Roles and Trust Model

SUDP realizes the secret-use interface required by Agent Secret Use (§3.1) by introducing one new protocol role: a *custodian* *T* that holds authority-bearing material as sealed state  $\Sigma$ , issues freshness, verifies *U*-rooted operation-bound grants, and performs the authorized secret-backed action within the bounds *U* signed. The custodian is the role through which SUDP implements the secret-use interface; it is not the interface itself. The custodian is a protocol role, not necessarily a deployment component: realizations include a local proxy, a hosted proxy, an OS-level credential mediator, an HSM- or TEE-backed service, or, in cooperative settings, the environment itself. The agentic specialization of the interface (how *R* is drawn to cover the LLM-context exposure surface, which secret-using layers of an agent enter the SUDP path, and how tool calls map to canonical operations) is the subject of §6.

**SUDP entities.** SUDP refines Agent Secret Use’s user  $U$  into the protocol *authorizer* and its autonomous delegate  $R$  into the protocol *requester*, and introduces a single new protocol role, the custodian  $T$ :

- $U$ : the *authorizer*, governing whether a secret-backed operation may proceed.
- $R$ : the *requester*, initiating an operation request, typically an agent.
- $T$ : the *custodian*, holding the sealed state  $\Sigma$ , validating authorization, redeeming operation-bound grants, and using protected authority material to perform the secret-backed operation; distinct from  $U$ .

We use *authorizer* rather than *user* as the label for  $U$  because the role is defined by its function (signing operation-bound grants), not by being human: the canonical deployment instantiates  $U$  as a human signing on a personal device, but the same role admits an agent acting under further-rooted authority in a chained delegation, with no change to the protocol’s mechanics. The external environment  $E$  is inherited from Agent Secret Use as the execution target; it is not a SUDP-speaking party and is not assumed to participate in the protocol beyond accepting its existing credential interface. The sealed state  $\Sigma$  is data managed by  $T$ , not a protocol-speaking party of its own.

**SUDP-specific trust assumptions.** Beyond the problem-level assumption of authorizer-controlled authorization (§3.1), SUDP’s architectural choices introduce two additional assumptions. *Restricted trust in the custodian:*  $T$  is trusted only to validate authorization, redeem grants, and transiently consume secret material during valid consumption; the requester is not trusted with secret access. *Sealed persistent state:*  $\Sigma$  is sealed such that the persistent state alone is insufficient for standalone recovery, and any authorized extraction leaves  $T$  only in recipient-protected form.

**Two execution patterns.** The protocol supports two execution patterns of the same abstraction. In *delegated execution*,  $R$  and  $U$  are distinct (an agent initiates an operation and the authorizer later approves it), so the authorization flow crosses parties. In *direct execution*,  $R = U$ , and the authorization flow collapses. The motivating agentic case is delegated execution; direct execution is a special case when initiation and authorization are co-located.

## 5.2 Key Hierarchy

SUDP uses the standard envelope-encryption pattern (Figure 3). The protected state  $M$  (the contents of  $\Sigma$  the protocol secures) is sealed once under a state-encryption key  $K$ ;  $K$  is in turn wrapped separately under each authorizer credential’s wrapping key  $W_c$ . This separation is deliberate: rotating  $K$  does not require interacting with every credential (each  $W_c$  simply rewraps the new  $K$ ), and registering a new credential does not require re-encrypting  $M$  (only a new  $[K]_c$  is appended). Formally,

$$C \leftarrow \text{Enc}_K(M), \quad M \leftarrow \text{Dec}_K(C),$$

and for each enrolled credential  $c$ ,

$$[K]_c \leftarrow \text{Wrap}_{W_c}(K), \quad K \leftarrow \text{Unwrap}_{W_c}([K]_c).$$

The persistent cryptographic state consists of  $C$  and  $\{[K]_c\}$ ;  $\{W_c\}$  defines the recoverability structure of  $K$ . By construction, the requester never receives  $K$ , any  $W_c$ , or any value from which either can be efficiently derived.

## 5.3 Authorized Operation

Rather than authorizing decryption in the abstract, SUDP authorizes a specific secret-backed operation together with its redemption binding and validity conditions. A well-formed authorization object encodes three aspects: (i) *authorization semantics*: what is approved; (ii) *redemption binding*: who may redeem and, if extracting, who receives; and (iii) *validity constraints*: how long the authorization is valid and how it resists replay and substitution. Each component corresponds to a distinct security obligation: the semantics must be reviewable at  $U$ , the binding must resist substitution, and the validity must resist replay. Freshness is supplied by the Phase II freshness token  $r$  (§5.6) rather than carried inside  $o$ ;  $o$  commits to  $r$  implicitly through the Phase II binding.



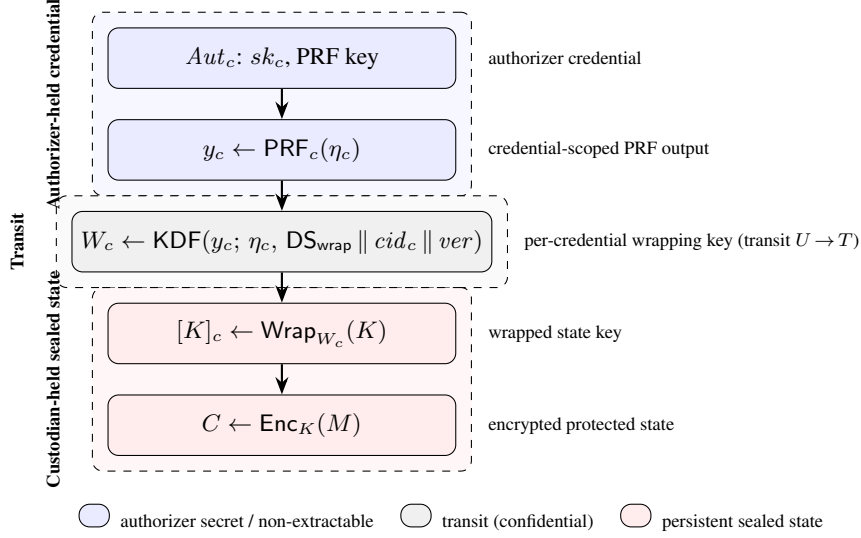


Figure 3: SUDP key hierarchy, organized in three trust zones. *Authorizer-held credential* material (blue) originates inside the authenticator  $Aut_c$  and never leaves it:  $sk_c$ , the PRF key, and the PRF output  $y_c$  stay there. *Transit* (gray) is the wrapping key  $W_c$  that  $U$  derives from  $y_c$  and sends to  $T$  under confidentiality. *Custodian-held sealed state* (red) is the persistent wrapping/encryption chain stored in  $\Sigma$ :  $[K]_c$  and  $C$ . Arrows trace the unlock chain downward; the figure shows one credential branch, with the acting-credential instance denoted  $W^*$  in Phase II (§5.6).

**Definition 4 (Authorized operation).** An authorized operation is a canonical protocol tuple

$$o := (\text{act}, \text{bind}, \text{valid}),$$

where  $\text{act} := (\text{type}, \text{target}, \text{scope})$ ,  $\text{bind} := (\text{redeemer}, \text{recipient})$ , and  $\text{valid} := (\text{expiry}, \text{multiplicity})$ . Here *type* specifies the semantic class of the secret-backed action (e.g., use, export, write, rotate, enroll, revoke); *target* identifies the protected object; *scope* carries canonicalized operation-specific constraints; *redeemer* identifies the party entitled to redeem; *recipient* identifies the intended recipient of any extracting delivery; *expiry* bounds the validity window; and *multiplicity*  $\in \{1, \infty\}$  selects the redemption budget within expiry: 1 consumes the grant on first redemption (intended for high-authority transaction-style actions);  $\infty$  admits any number of redemptions within expiry (intended for  $U$ -declared low-risk repeat patterns such as inbox reads). Freshness for the Phase II verification is enforced separately, via the single-use token  $r$  issued at Phase II.1 and consumed at Phase II.3.

We reserve specific terms: a *grant*  $G$  is the protocol artifact representing  $U$ 's authorization of  $o$ ; it commits to  $o$  and conveys for redemption both the authorization signature and the acting credential's wrapping key (the full structure is in §5.6). The wrapping key is secret:  $G$  is therefore not a public bearer token, and the  $U \rightarrow T$  leg carrying it requires end-to-end confidentiality. A *redeemed grant*  $\rho$  is  $G$ 's custodian-internal post-validation form. SUDP binds  $G$  and  $\rho$  to the same  $\beta$  commitment in Phase II so that the artefact  $T$  redeems matches the one  $U$  authorized.

SUDP's cryptographic binding attaches to the canonical bytes of  $o$ . Two deployment obligations anchor those bytes at the endpoints of the authorization; AV and OB hold when both are met.

**Render at  $U$ .**  $\text{Render}(o)$  is deterministic, complete, and confined to  $U$ 's trusted client, so  $U$  approves exactly what  $H(o)$  commits.

**Execution at  $E$ .** The action performed at  $E$  is a deterministic function of  $o$ , the material  $T$  unwraps, and any auxiliary content that  $o$  commits by hash. No authority-relevant input enters between  $U$ 's authorization and the action.

## 5.4 Cryptographic Primitives

We fix primitive interfaces here; concrete algorithm choices appear in §7.

- $H$ : collision-resistant hash.
- $KDF(ikm; salt, info)$ : HKDF-style extract-then-expand with explicit domain separation in  $info$ .
- $(Enc, Dec), c \leftarrow Enc_k(m; ad)$ : IND-CCA AEAD with associated-data authentication.
- $(Sig, Vrfy)$ : EUF-CMA secure.
- $(Encap, Decap)$ : IND-CCA2 secure.
- CSPRNG: cryptographically secure randomness source.
- $(Wrap, Unwrap), E \leftarrow Wrap_W(K)$ : key-wrap interface specialized to key material.
- $PRF_c$ : authenticator-bound pseudo-random function, invoked only inside the module  $Aut_c$  (specified below).

The  $Wrap$  interface binds wrapping context through the derivation of  $W$  rather than through a separate parameter: because  $W$  is produced by a domain-separated KDF keyed to the credential, salt, and version (§5.2), any attempt to unwrap under a  $W$  derived from different context material fails by construction, and the interface accommodates both associated-data-free wrap constructions (such as AES-KW/KWP) and AEAD-as-wrap profiles that additionally authenticate the same domain-separation labels.

**Authenticator module  $Aut_c$ .** Each authorizer-side credential  $c$  is modeled as a tamper-resistant module  $Aut_c$  with non-extractable internal keys. Under a single  $U$ -verified invocation,  $Aut_c$  produces one signature  $\sigma \leftarrow Sig_{sk_c}(\mu)$  over an externally supplied challenge  $\mu$ , and evaluates  $PRF_c$  on one or more caller-supplied salts  $\eta_i$ , returning the outputs  $y_i \leftarrow PRF_c(\eta_i)$ . The number of PRF evaluations admissible per invocation is a parameter of  $Aut_c$ 's interface; the rotation-class operations of §5.7 require at least two. The signing key and the PRF key never leave the module. Enrollment releases public material  $(cid_c, pk_c)$ . Every  $info$  or  $ad$  argument carrying a domain-separation label  $DS_*$  uses a stable, context-disjoint string; labels are pairwise disjoint and no label is ever reused. The protocol equations explicitly name  $DS_{bind}$  and  $DS_{wrap}$ ; concrete profiles may use additional disjoint labels for sealing, delivery, and transport without affecting the abstract contract.

**Canonical serialization.**  $H(o)$  presumes a deterministic encoding of the structured value  $o$  that is also identical at  $U$  and  $T$ . A concrete profile must fix this serialization (for instance, a canonical CBOR or RFC 8785 JCS encoding of the (act, bind, valid) fields) and enforce it at both endpoints. Ambiguity in the encoding can defeat operation binding without violating collision resistance of  $H$ : two semantically distinct descriptors may serialize to the same bytes (so that an adversary substitutes  $o' \neq o$  with the same  $H$ -input), or the same descriptor may serialize differently at  $U$  and  $T$  (so that  $U$  authorizes an operation whose canonical form at  $T$  differs from the one Render presented at authorization time). Both failures collapse OB regardless of  $H$ 's strength. The specific serialization is a profile parameter, not a protocol choice.

Pairwise communication runs over a pre-established authenticated channel; transport is assumed, not re-specified. Confidentiality is required on the  $U \rightarrow T$  leg (which carries  $W^*$ ) and on the final response leg (which may carry the delivery artifact  $\pi$  or a use-result  $\rho_{out}$ ). This requirement is on the logical  $U \rightarrow T$  leg end-to-end and is not in general satisfied by hop-by-hop transport security: deployments in which an intermediary (a TLS-terminating relay, L7 proxy, or load balancer) can inspect plaintext between  $U$  and  $T$  must supply confidentiality end-to-end at the application layer (e.g., the cross-device HPKE profile of §7.2) rather than relying on chained TLS hops alone. Integrity is required on every leg. The  $T \rightarrow U$  conveyance (Phase II.1) carries only public material  $(o, r, \{(cid_c, \eta_c)\})$  and does not require confidentiality; its only protection need is against denial-of-service tampering, which is detected at redemption through the signature on  $\beta$ .

## 5.5 Phase I: Setup

Setup initializes the persistent cryptographic state required by the remaining phases. Its structural outcome is that, after the setup phase completes and transient values are erased, no persistent state held by any single party contains both  $K$  and any wrapping key  $W_c$ , and every enrolled credential is on an independent authorization path to  $K$ . The requester  $R$  is not involved.

**I.1 Credential enrollment.**  $U$  registers one or more authenticator credentials with  $T$ . Each credential creation generates  $Aut_c$ 's internal PRF key and signing key (neither of which ever leaves the module) and releases a credential identifier  $cid_c$  together with the public verification key  $pk_c$ :

$$(cid_c, pk_c) \leftarrow \text{Enroll}(Aut_c).$$

$T$  records the registration map  $\text{Reg} := \{cid_c \mapsto pk_c\}_{c \in \text{Creds}}$ , which Phase II.3 will use to verify authorization evidence.

**I.2 Protected-state sealing.**  $T$  generates a fresh state-encryption key and seals the initial protected state:

$$K \xleftarrow{\$} \text{CSPRNG}, \quad C \leftarrow \text{Enc}_K(M_0; ver).$$

For each enrolled credential  $c$ ,  $U$  and  $T$  jointly derive a credential-specific wrapping key through a single  $U$ -verified invocation of  $Aut_c$ . The authenticator evaluates its PRF under a freshly sampled salt  $\eta_c$  to produce  $y_c$ ;  $U$  derives the wrapping key  $W_c$  and transmits it to  $T$ , which produces the wrapped entry  $[K]_c$ :

$$\begin{aligned} \eta_c &\xleftarrow{\$} \text{CSPRNG}, \quad y_c \leftarrow \text{PRF}_c(\eta_c), \\ W_c &\leftarrow \text{KDF}(y_c; \eta_c, \text{DS}_{\text{wrap}} \parallel cid_c \parallel ver), \\ [K]_c &\leftarrow \text{Wrap}_{W_c}(K). \end{aligned}$$

The authenticator-internal PRF key never leaves  $Aut_c$ ; only  $W_c$  crosses the  $U \rightarrow T$  channel, and is zeroized at  $T$  once  $[K]_c$  is persisted. The domain-separation info binds  $cid_c$  and  $ver$ , so derivations for distinct credentials and distinct wrapping epochs remain independent even when the same authenticator hosts several credentials.

**I.3 Freshness state.**  $T$  initializes a bounded single-use pool  $S$  of freshness tokens  $(r, \tau_r)$ . Each token is issued and consumed exactly once in Phase II and has short lifetime  $\tau_r$  (on the order of minutes), which bounds the set of outstanding authorizations.

**I.4 Output.** Phase I leaves the persistent state

$$\Sigma_0 := (C, \{(cid_c, \eta_c, [K]_c)\}_{c \in \text{Creds}}, \text{Reg}, ver).$$

Two invariants close the phase. *Persistent-state separation*: after transient values from I.2 are zeroized, no persistent state held by any party contains both  $K$  and any  $W_c$ ;  $U$ 's client stores no extractable persistent key material, the authenticator retains non-extractable credential keys, and  $\Sigma_0$  alone is insufficient for plaintext recovery of  $M_0$ . *Uniform recoverability*: every enrolled credential  $c$  can reach  $K$  via one  $U$ -verified invocation of  $Aut_c$  together with  $\Sigma_0$  alone.

## 5.6 Phase II: Authorization Grant

The grant phase converts an authorization decision into a  $U$ -signed grant  $G$  that commits cryptographically to  $o$ ;  $T$ 's post-validation record of  $G$  is the redeemed grant  $\rho$  (Figure 4).

**II.1 Grant request.**  $R$  initiates by requesting authorization for  $o = (\text{act}, \text{bind}, \text{valid})$ , with  $\text{bind.redeemer} = T$  and, for extracting operations,  $\text{bind.recipient}$  set to the intended recipient's public key. The descriptor  $o$  is the cryptographic contract between  $T$  and  $U$ :  $T$  binds to  $H(o)$  in the channel binding  $\beta$ , and  $U$  reviews  $\text{Render}(o)$  before signing.  $T$  issues a fresh freshness token  $r \xleftarrow{\$} \text{CSPRNG}$ , stores  $(r, \tau_r)$  in  $S$ , and prepares the conveyance payload  $(o, r, \{(cid_c, \eta_c)\}_{c \in \text{Creds}})$ . The  $(cid_c, \eta_c)$  entries are public material:  $cid_c$  is a credential identifier,  $\eta_c$  is the current per-credential PRF salt, and both are needed by  $U$  to invoke the correct authenticator. The conveyance from  $T$  to  $U$  is a logical authenticated channel; its realization (a direct  $T \rightarrow U$  session, or a broker hop carried through  $R$ ) is a deployment dimension treated in §6 and does not enter the protocol's obligations. Confidentiality is not required on this leg:  $o$  is not sensitive, and any in-transit substitution will be detected at redemption against the signature on  $\beta$  (II.4). In direct execution ( $R = U$ ), the conveyance collapses into the same session.

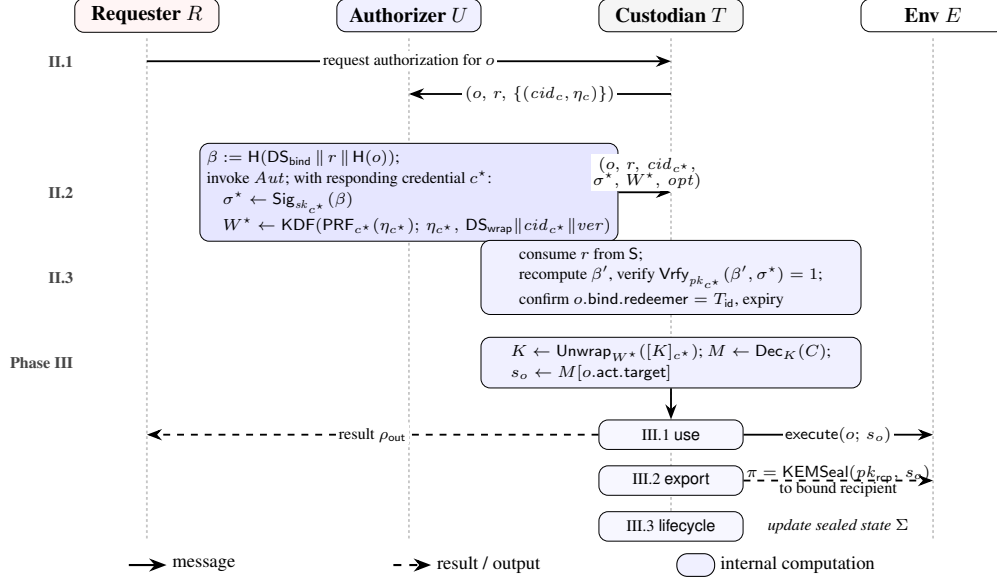


Figure 4: End-to-end SUDP flow across requester  $R$ , authorizer  $U$ , custodian  $T$ , and environment  $E$ . Phase II issues a  $U$ -signed grant  $G$  committing to  $o$ ; Phase III unwraps  $K$  with  $W^*$  and dispatches on  $o.act.type$  to use, export, or lifecycle. The  $U \rightarrow T$  leg carries  $W^*$  under confidentiality.

**II.2 Authorizer signing.**  $U$  renders  $o$  inside its trusted boundary and decides whether to approve. On approval,  $U$  selects an acting credential  $c^* \in \text{Creds}$  and, in a single  $U$ -verified invocation of  $Aut_{c^*}$ , produces a signature over  $\beta$  and a PRF-derived value for  $\eta_{c^*}$ .

*Operation- and freshness-bound assertion.*  $U$  computes the binding

$$\beta := H(\text{DS}_{\text{bind}} \parallel r \parallel H(o)),$$

passes  $\beta$  to  $Aut_{c^*}$  as the signing challenge, and obtains

$$\sigma^* \leftarrow \text{Sig}_{sk_{c^*}}(\beta).$$

The binding  $\beta$  commits to the session (via  $r$ ), the domain-separated context label, and the operation (via  $H(o)$ ): any change to  $o$  between II.2 and II.3 causes the  $\beta'$  recomputed at redemption to diverge from the  $\beta$  signed into  $\sigma^*$ .

*Derived wrapping material.* The same invocation evaluates  $Aut_{c^*}$ 's PRF and derives the per-credential wrapping key for  $c^*$ :

$$y^* := \text{PRF}_{c^*}(\eta_{c^*}), \quad W^* \leftarrow \text{KDF}(y^*; \eta_{c^*}, \text{DS}_{\text{wrap}} \parallel \text{cid}_{c^*} \parallel \text{ver}).$$

For rotation-class operations (§5.7),  $U$  additionally samples a fresh next-salt  $\eta_{c^*}^{\text{next}}$ , places it inside  $o.act.scope$  so that  $\beta$  commits to it via  $H(o)$ , and derives  $W_{\text{next}}^*$  analogously from  $\text{PRF}_{c^*}(\eta_{c^*}^{\text{next}})$  in the same invocation ( $Aut_{c^*}$  admits two PRF evaluations per invocation, §5.4).

$U$  transmits the grant

$$G := (o, r, \text{cid}_{c^*}, W^*, \sigma^*, \text{opt}),$$

to  $T$  over an end-to-end authenticated and confidential channel; here  $\text{opt} = (W_{\text{next}}^*)$  appears only for rotation-class  $o$ . Channel confidentiality is required end-to-end on this leg: any party that observes  $W^*$  can unwrap  $[K]_{c^*}$ , and the confidentiality of  $W^*$  is not protected by the signature. The realization may be a direct  $U \rightarrow T$  session or an opaque relay through any intermediary (including  $R$ ) under the cross-device envelope of §7.2, which keeps  $W^*$  hidden from the relay; the two carrier topologies are catalogued in §7.3. The asymmetry between the  $T \rightarrow U$  conveyance (public payload, no confidentiality needed) and the  $U \rightarrow T$  transmission (carries  $W^*$ , end-to-end confidentiality required) is a security-critical part of the channel model.

**II.3 Grant redemption.** On receipt of  $G$ ,  $T$  validates the grant by:

1. consume  $(r, \cdot)$  from  $S$  (reject if absent or expired);
2. check  $o.\text{valid.expiry}$ ,  $o.\text{bind.redeemer}$ , and act-specific shape conditions (rotation-class operations require  $W_{\text{next}}^*$  in  $opt$ ; extracting operations require  $o.\text{bind.recipient}$ );
3. recompute  $\beta' := H(\text{DS}_{\text{bind}} \parallel r \parallel H(o))$  from the received  $o$ ;
4. verify  $\text{Vrfy}_{\text{Reg}[cid_{c^*}]}(\beta', \sigma^*) = 1$ .

$r$  is consumed regardless of which step fails, preventing reuse of the same freshness token. As standard hygiene against timing side channels, the comparison inside  $\text{Vrfy}$  is performed in constant time. On success,  $T$  emits the redeemed grant

$$\rho := (o, cid_{c^*}, W^*, opt).$$

$\rho$  is custodian-internal:  $R$  never sees it.  $r$ 's single-use consumption at II.3 prevents the same  $(o, r, \sigma^*)$  triple from being re-redeemed;  $\rho$ 's Phase III lifecycle is governed by  $o.\text{valid}$  (§5.3).

**II.4 Anti-substitution.** Substitution, tampering, or injection of a modified  $o'$  between approval and redemption changes  $\beta'$  and fails verification at II.3 under EUF-CMA of  $\text{Sig}$ ; together with the trusted rendering of  $\text{Render}(o)$  inside  $U$ 's boundary, this ensures the  $o$  that  $T$  validates is the  $o$  that  $U$  reviewed.

**Batch extension.** The grant generalizes from a single operation to an ordered list  $\text{ops} := (o_1, \dots, o_n)$  authorized under one signature by substituting  $H(\text{ops})$  for  $H(o)$  in the binding,

$$\beta := H(\text{DS}_{\text{bind}} \parallel r \parallel H(\text{ops})),$$

where  $H(\text{ops})$  is computed over a canonical concatenation of the  $o_i$ .  $U$ 's trusted rendering obligation extends to  $\text{Render}(\text{ops})$  as a single decision surface, and  $T$  validates each  $o_i$  under the II.3 obligations. The single-operation case is the  $n = 1$  instance; the cryptographic core, redemption sequence, and security properties are unchanged.

## 5.7 Phase III: Consumption

Consumption executes  $o$  against the sealed persistent state under  $\rho$ , in a form determined by  $o.\text{act.type}$ . SUDP distinguishes three semantic cases of Phase III (not deployment APIs): *non-extracting use* (III.1) spends the secret inside  $T$  and releases only the authorized result form to  $R$ ; *recipient-protected export* (III.2) releases protected material only to the recipient named in  $o.\text{bind}$  and is ASU-preserving only when that recipient lies outside  $R$ 's trust boundary; and *state-update / lifecycle* operations (III.3) mutate protected state and are coupled to rotation. All three begin with the same access step (Figure 5).

**III.0 Access to protected state.** We write  $s_o := M[o.\text{act.target}]$  for the authority-bearing service secret selected by an accepted operation (e.g., the API key, OAuth token, or signing key referenced by  $o.\text{act.target}$ );  $s_o$  is the only secret  $T$  materializes for  $o$ , and the symbol  $s$  (Definition 1) is its abstract counterpart. Given  $\rho$ ,  $T$  uses the acting credential's wrapping key  $W^*$  carried in  $\rho$  to unwrap  $K$ , decrypts  $M$ , and selects  $s_o$ :

$$K \leftarrow \text{Unwrap}_{W^*}([K]_{c^*}), \quad M \leftarrow \text{Dec}_K(C).$$

The  $\text{Unwrap}$  succeeds only under the  $W^*$  bound to the matching salt, credential identifier, and version at Phase I.2; the  $\text{Dec}$  call binds its AEAD associated data to the protected-state context; any failure aborts the phase without releasing plaintext.  $\rho$ 's lifetime in  $S$  is governed by  $o.\text{valid}$  (§5.3). On success,  $T$  holds  $K$  and  $M$  within its trusted execution boundary and dispatches on  $o.\text{act.type}$ .

**III.1 Non-extracting consumption** (type = use).  $T$  applies the authorized action to  $M$  inside its own boundary and never releases secret material from  $M$  to  $R$ . We write  $\text{Release}(o) = \rho_{\text{out}}$  for the result form returned to  $R$ . The protocol guarantees that  $\text{Release}(o)$  contains no plaintext element of  $M$ ; the substantive content of  $\text{Release}(o)$  is released under  $o.\text{act.scope}$ , since the downstream environment's response may itself carry sensitive or authority-amplified data. For signing operations specifically, the scope must bind the exact message or transaction digest, the intended verifier/domain/audience, and the replay domain, so that a  $\text{Release}(o)$  signature is not usable outside the authorized context.

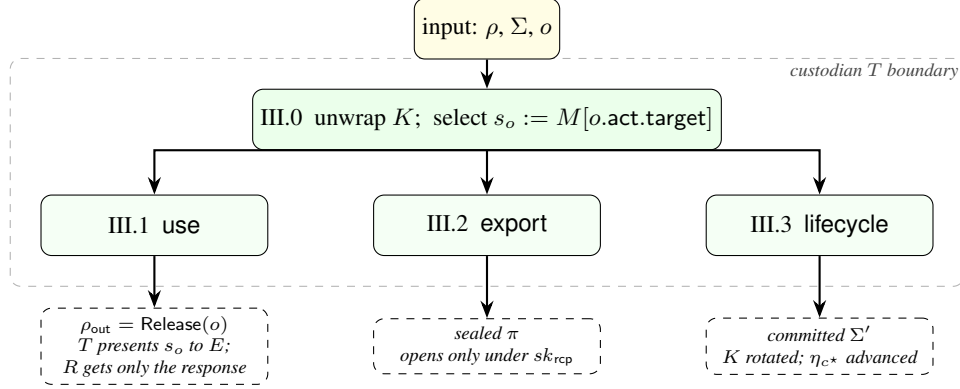


Figure 5: Phase III dispatch inside the custodian  $T$  boundary. III.0 unwraps  $K$  and selects the authority-bearing service secret  $s_o := M[o.act.target]$ ; dispatch on  $o.act.type$  then selects the consumption mode. *Use*:  $T$  may present  $s_o$  to  $E$ 's native interface but never to  $R$ ;  $R$  receives only  $\text{Release}(o) = \rho_{out}$ . *Export*:  $T$  emits a sealed delivery  $\pi$  that opens only under  $sk_{rcp}$ . *Lifecycle*:  $T$  commits a new sealed state  $\Sigma'$  with rotated  $K$  and advanced authenticator-credential salt  $\eta_{c*}$ . In all three modes, reusable authority never crosses the  $T$  boundary toward  $R$ .

**III.2 Recipient-protected extracting delivery** (type = export). Some operations inherently require extraction: migrating a secret between devices, provisioning a downstream service, or handing off to a specialized sub-agent. SUDP admits these under the constraint that the extracted value leaves  $T$  only in a form cryptographically protected for a named recipient. For ASU-preserving execution, a well-formed export operation requires  $\text{bind.recipient}$  to denote a party outside  $R$ 's trust boundary; if  $U$  authorizes an export with  $\text{bind.recipient} = R$ , the operation is a deliberate ownership-transfer that falls outside SUDP's CRC and ASU non-disclosure guarantees and must be surfaced as such at authorization time. Given  $pk_{rcp}$  from  $o.bind$ ,  $T$  uses a key-encapsulation mechanism to produce an ephemeral encapsulated key, derives a delivery key bound to the specific operation via  $H(o)$ , and seals the target:

$$\begin{aligned} (K_d, ct_d) &\leftarrow \text{Encap}(pk_{rcp}), \\ k_d &\leftarrow \text{KDF}(K_d; \perp, H(o)), \\ \delta &\leftarrow \text{Enc}_{k_d}(s_o; H(o)). \end{aligned}$$

$T$  emits the delivery artifact  $\pi := (ct_d, \delta)$ . Only the holder of  $sk_{rcp}$  can Decap to recover  $K_d$ , rederive  $k_d$ , and open  $\delta$ . Binding both the KDF info and the AEAD associated data to  $H(o)$  prevents an adversary from substituting a captured  $\pi$  into the transcript of a different authorized operation: any such substitution would change  $H(o)$  and fail the AEAD integrity check. When the recipient is a third party,  $R$  may act as a relay and gains no additional capability because  $\pi$  transits  $R$  only in sealed form.

**III.3 State-update / lifecycle consumption** (type  $\in \{\text{write, rotate, enroll, revoke}\}$ ). Operations that mutate  $M$  (writes, pure  $K$ -rotations, and lifecycle events such as credential enrollment or revocation) are structurally coupled to key rotation. A single consumption applies  $o$  to  $M$  to obtain  $M'$ , samples a fresh state key  $K'$  under which  $M'$  is re-sealed, and replaces the acting credential's wrapping material using  $W_{next}^*$  carried in  $opt$ :

$$\begin{aligned} K' &\xleftarrow{\$} \text{CSPRNG}, \\ C' &\leftarrow \text{Enc}_{K'}(M'; ver), \\ E'_{c*} &\leftarrow \text{Wrap}_{W_{next}^*}(K'). \end{aligned}$$

The update must be atomic as a protocol invariant: either  $(C, \{[K]_c\})$  is replaced by  $(C', \{E'_c\})$  in its entirety or neither is written. As an implementation requirement for atomicity, the new wrapping material  $\{E'_c\}$  must be persisted durably before the old entries are retired; any crash between the two steps must leave the old  $(C, \{[K]_c\})$  recoverable, never a state in which both sides have been partially overwritten. Violating this order leaves the only decryption path to  $K$  unreachable and produces an

unrecoverable  $\Sigma$ . Multi-path recoverability requires rewrapping  $K'$  under each  $W_c$  for  $c \neq c^*$ ; the structure of this rewrap is part of the security argument and is addressed in §5.8.

**Extensibility of the dispatch vocabulary.** The vocabulary  $\{\text{use}, \text{export}, \text{lifecycle}\}$  covers the canonical consumption modes; concrete profiles MAY define additional dispatch types provided each new type (i) preserves the channel binding  $\beta = H(\text{DS}_{\text{bind}} \parallel r \parallel H(o))$  and the grant-validation steps of II.3 unchanged, (ii) inherits or explicitly restates its AV/OB/UB stance, and (iii) specifies its non-disclosure stance against  $R$ , stating what plaintext-bearing material, if any, leaves  $T$ . Profile-defined dispatch types remain within the abstract protocol when they meet these conditions and do not require modification of Phases I or II. Extensions that weaken non-disclosure relative to the canonical modes are admissible at the profile level only as deliberate trade-offs and must be surfaced as such, both at authorization time to  $U$  and in the profile’s stated security claims.

## 5.8 Rotation and Forward Secrecy

Rotation is a *protocol invariant* of SUDP, not an optional discipline: every SUDP-conformant deployment advances both the wrapping epoch on every state-update and, on a deployment-defined cadence, the underlying authority-bearing secret at  $E$  together with revocation of the retired secret. The first part is enforced by Phase III.3 below; the second part is mandated by the protocol’s type = rotate operation class (Phase III.3, §5.7) and its compliance requirement that authority-bearing secrets never persist beyond their rotation epoch. This section specifies the wrapping-side mechanism; the authority-side cadence is a deployment policy parameter, not a rotation-skipping option.

**Rotation triggers.** Every state-update consumption (III.3) generates a fresh  $K'$  and rotates the acting credential’s salt  $\eta_{c^*} \rightarrow \eta_{c^*}^{\text{next}}$ . Two specialized rotation modes arise as sub-cases:

- **Rewrap** preserves the authority-bearing plaintext entries in  $M$  but reseals state under a fresh  $K'$  and fresh acting-credential wrapping material. Its purpose is credential hygiene: advancing a credential’s salt independently of write traffic, or migrating it to new domain-separation parameters. Formally, rewrap is a III.3 execution in which the non-peer contents of  $M$  are unchanged and the  $\text{Peer}[cid_c]$  entry is updated as described below.
- **Full rotation** rotates  $K$  and the acting credential’s  $W$ , and rewraps all other credentials under the same  $K'$ . Formally, this is the ordinary III.3 execution with  $o.\text{act.type} = \text{rotate}$  and  $M' = M$ .

Coupling rotation to every write tightens the wrapping-epoch forward-secrecy profile without introducing a new ceremony: any adversary who has transiently obtained pre-rotation key material is locked out of wrapped state produced after the next write. Authority-level forward secrecy additionally requires that retired authority-bearing secrets at  $E$  be revoked on the deployment’s rotation cadence; without that revocation, compromise of the current epoch confers the same authority as compromise of any prior epoch and the wrapping-side forward secrecy is meaningless against  $E$ .

**Forward secrecy.** Let  $\text{rot}_{c^*}(t)$  denote the most recent time  $t' \leq t$  at which credential  $c^*$  performed a state-update. The following claim is instantiated by SUDP’s per-write discipline.

**Lemma 1 (Per-authenticator-credential wrapping forward secrecy).** *For any credential  $c^*$  and any  $t' < \text{rot}_{c^*}(t)$ , compromise of the tuple  $(y_{c^*}^{(t')}, W_{c^*}^{(t')})$  does not enable recovery of  $K$  or  $M$  from  $\Sigma_t$ , under the PRF security of  $\text{Aut}_{c^*}$ , the key-hiding property of KDF, and the key-unforgeability of Wrap.*

*Proof sketch.* At  $\text{rot}_{c^*}(t)$ , the next-salt  $\eta_{c^*}^{\text{next}}$  is sampled inside the rotation invocation from a fresh CSPRNG draw, independent of all prior salts. Under the PRF security of  $\text{Aut}_{c^*}$ , the PRF output on  $\eta_{c^*}^{\text{next}}$  is indistinguishable from random given any number of PRF outputs under distinct prior salts; hence the compromised  $y_{c^*}^{(t')}$  carries no information about the post-rotation  $y_{c^*}^{\text{new}}$ . The domain-separated KDF propagates this indistinguishability to  $W_{c^*}^{\text{new}}$ . Finally, the wrapped entry in  $\Sigma_t$  is under  $W_{c^*}^{\text{new}}$ , so no efficient adversary holding only pre-rotation state can unwrap.  $\square$

Lemma 1 is per-credential and activates only after a credential rotates; the residual gap for dormant credentials is catalogued as L3 in §9.6.

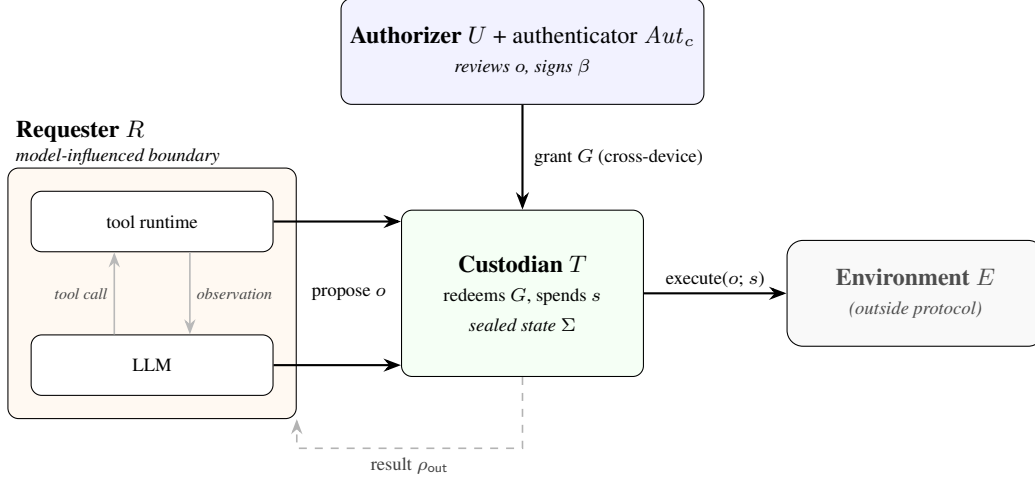


Figure 6: **The agentic secret-use interface.** An agent uses secrets at two layers, both inside  $R$ : the LLM-API call to a model provider (typically authenticated with an operator-owned key, outside ASU and not routed through SUDP) and tool-call execution. SUDP applies to a tool call when its execution would exercise authority-bearing material (Definition 1) enrolled by  $U$  at an external environment  $E$ ; other tool calls pass through unmodified. For an in-scope call,  $R$  proposes operation  $o$ ;  $U$  (with authenticator  $Aut_c$ , typically on a separate device) reviews  $o$  and signs  $\beta$ ; the custodian  $T$  redeems the grant  $G$  and spends  $s$  only for the accepted  $o$  at  $E$ .  $R$  receives only the release form  $\rho_{out}$ ; reusable authority never crosses  $R$ 's boundary.

**Default rewrap: in-state peer map.** In multi-credential SUDP deployments, Phase III.3 by credential  $c^*$  must rewrap  $K'$  under  $\{W_c\}_{c \neq c^*}$ . The default construction adopts an *in-state peer map*

$$\text{Peer} := \{cid_c \mapsto W_c\}_{c \in \text{Creds}} \subseteq M,$$

which III.0's decryption of  $C$  transiently exposes to  $T$ . Under this policy, III.3 completes by rewrapping  $E'_c \leftarrow \text{Wrap}_{W_c}(K')$  for each  $c \neq c^*$ , updating  $\text{Peer}[cid_{c^*}] := W_{new}^*$ , and sealing:

$$\Sigma' := (C', \{(cid_c, \eta'_c, E'_c)\}_{c \in \text{Creds}}, \text{Reg}, ver),$$

with  $\eta'_{c^*} = \eta_{c^*}^{\text{next}}$  and  $\eta'_c = \eta_c$  for  $c \neq c^*$ .  $\rho$  is marked consumed; as an implementation hardening note,  $T$  zeroizes  $K, K'$ , all  $W$  values, and  $M$  after persistence. The cross-credential exposure entailed by the in-state peer map is catalogued as L4 in §9.6; alternative rewrap policies (all-present rotation, append-only catch-up chain) are discussed in §8.

**Lifecycle extensions: enrollment and revocation.** Credential addition and removal are ordinary III.3 executions with  $o.act$  naming the credential affected and  $M'$  encoding the change. *Enrolling* a credential  $c^+$  proceeds in two coupled  $U$ -verified invocations bundled into one operation: an enrollment invocation of  $Aut_{c^+}$  that yields  $(cid_{c^+}, pk_{c^+}, \eta_{c^+}, W_{c^+})$ , and the acting-credential rotation invocation of  $Aut_{c^*}$  that authorizes the write.  $T$  computes  $[K]_{c^+} \leftarrow \text{Wrap}_{W_{c^+}}(K')$ , adds  $(cid_{c^+}, \eta_{c^+}, [K]_{c^+})$  to  $\Sigma'$ , and updates  $\text{Peer}[cid_{c^+}] := W_{c^+}$ . *Revocation* removes the target credential from  $\text{Reg}$ , from the wrapping set, and from  $\text{Peer}$ , and reseals  $M'$  under a fresh  $K'$ . Neither operation requires machinery beyond what has already been specified.

## 6 Secret-Use Interfaces for Agentic Systems

Agentic systems pose two structural questions for Agent Secret Use that the protocol-level definitions do not directly answer: *where does the requester boundary end* in a system whose actions are shaped by an external LLM, and *which secret-backed actions inside the agent need to enter the SUDP path*. Figure 6 shows the structural answer. §6.1 draws  $R$  wide enough to cover the LLM-context exposure surface; §6.2 separates LLM-API from tool-call secret use and identifies which subset is in scope; §6.3 specifies the adapter that maps an in-scope tool call to a canonical operation.



## 6.1 The Agentic Requester Boundary

The defining property of an agentic system is that the LLM’s context determines its actions. Anything placed in that context has effectively left the local trust domain: production LLMs are typically third-party services that observe every token sent to them, may log inputs, and may train on them under operator-side policy. The same applies to retrieval contents, tool outputs, and scratchpad rewrites that re-enter context as observations. Treating only the LLM-runtime process as  $R$  is therefore unsound; the secret-relevant exposure surface is the union of everything that can reach the LLM context, including the tool scheduler, tool clients (MCP-style brokered runtimes among them), agent-visible memory, and any code whose behavior an attacker-controlled observation can influence.  $R$  in an agentic deployment is this full model-influenced execution boundary. The custodian  $T$  and  $U$ ’s authenticator remain outside it.

## 6.2 Where SUDP Applies in an Agentic System

An agent uses secrets at two distinct layers, and SUDP applies to only one.

**LLM-API layer.** The LLM-API call is itself secret-backed: the agent authenticates to a model provider with an API key. When that key is operator-owned and bills against the platform’s account, it represents the operator’s authority over the model provider, not the user’s authority over an external environment; it lies outside the Agent Secret Use problem of §3.1 and does not require a per-call SUDP authorization. A deployment that intentionally models the model-provider key as user authority (a BYOK product, for instance) may route it through SUDP like any other  $U$ -enrolled authority-bearing material; the default position is that LLM-layer authentication is platform-internal.

**Tool-call layer.** Tool calls reach external environments  $E$ : mail services, code hosts, cloud APIs, payment rails. A tool call enters SUDP’s scope when its execution would exercise authority-bearing material (Definition 1) enrolled by  $U$  at  $E$ , such as a Gmail token, a GitHub token, or a cloud credential. For such calls,  $R$  proposes a canonical operation  $o$  (§6.3),  $U$  authorizes  $o$ , and  $T$  spends the underlying secret only for the accepted  $o$ . Tool calls that do not touch authority-bearing material (a public web search, an unauthenticated arithmetic helper, a local file read) lie outside scope and pass through unmodified.

**Scope: secrets, not behavior.** SUDP protects authority-bearing material and the authorization channel; it does not constrain what an agent may attempt. A prompt-injected agent can still propose arbitrary operations to  $T$ , but it cannot read  $s$  or any reusable artifact derived from  $s$  (CRC), and any operation  $T$  executes is one  $U$  explicitly authorized (AV, OB, UB). Behavioral defenses (runtime monitors, policy DSLs, taint tracking, sandboxes) reason about agent actions rather than the credentials those actions consume; §4 positions them as composable with SUDP rather than substitutive.

## 6.3 Adapting a Tool Call to a Canonical Operation

A tool-specific *adapter* maps an in-scope tool call to a canonical operation  $o$ . Per §5.3, the adapter has four responsibilities: (i) identify the authority-relevant fields of the call; (ii) canonicalize them into  $o$ ; (iii) produce the trusted rendering  $\text{Render}(o)$  shown to  $U$ ; and (iv) define the deterministic execution mapping from accepted  $o$  to the native call at  $E$ . The first is per-tool (a schema design choice); the remaining three reuse the protocol-level invariants of §5.3.

Schemas are typically terse. Three illustrative cases:

- **GitHub.** type=use; target= GitHub token; scope=(method, repo, endpoint, branch, body\_hash).
- **Email.** type=use; target= mailbox credential; scope= (send, to, cc/bcc, subject\_hash, body\_hash, attachment\_hashes).
- **Cloud API.** type=use; target= cloud credential; scope= (service, region, action, resource\_arn, request\_body\_hash, idempotency\_key).

These are schemas only; adapter implementation, a declarative service registry, or a manual specification are realization choices outside the protocol abstraction. The completeness condition of §5.3

(formalized as P1 in §9) fixes the obligation: any field that can affect the authority exercised at  $E$  must be in the schema, or it falls outside SUDP’s operation-binding guarantee.

## 7 Standard Construction

§6 specialized the secret-use interface to agentic systems. This section gives the standards-based cryptographic instantiation of the protocol primitives: each abstract interface of §5 is fixed to a standardized component. §7.1 covers primitive selection layer by layer; §7.2 instantiates the Phase II authenticated channel (§5.6) for the cross-device case when  $U$  and  $T$  are not co-located. The construction introduces no new cryptographic assumption.

### 7.1 Primitives

**Authenticator  $Aut_c$ .** The abstract protocol requires a tamper-resistant module  $Aut_c$  that (i) signs a challenge under user verification and (ii) emits an authenticator-bound pseudo-random value scoped per credential. We instantiate  $Aut_c$  with WebAuthn credentials using the PRF extension [Hodges et al., 2021, FIDO Alliance, 2025].

The abstract  $\sigma \leftarrow \text{Sig}_{sk_c}(\beta)$  interface is realized by a WebAuthn *assertion* produced with challenge  $= \beta$ , using ES256/P-256 as the profile-selected COSE signing algorithm. The corresponding Vrfy check covers the assertion signature together with the assertion’s `rpId` hash, origin, user-verification flag, and credential-ID binding; a signature-verification success without these structural checks does not satisfy the Sig interface in this profile.

The authenticator-bound pseudo-random value is realized by the WebAuthn PRF extension, backed by CTAP `hmac-secret` where available, with a per-credential salt; credentials or assertions whose response does not contain the PRF output are rejected by the profile. Two compliance implications follow: (a) binding PRF output to a passkey means deletion of the passkey implies loss of access to any data encrypted under the derived key; (b) per-credential PRF salts support domain separation across credentials.

**Key schedule.** The abstract protocol requires a KDF that supports extract-then-expand semantics and domain separation. We instantiate KDF with HKDF-SHA-256 [Krawczyk and Eronen, 2010], following NIST SP 800-56C key-derivation guidance [Barker et al., 2020]. Domain separation is achieved by encoding a disjoint  $DS_*$  label in the `info` argument of each derivation; this is sufficient to ensure that distinct derivations produce independent keys under the hash-function random-oracle idealization.

**Sealing and key wrapping.** Enc/Dec is instantiated with a standard AEAD scheme such as AES-GCM [Dworkin, 2007] or ChaCha20-Poly1305 [Nir and Langley, 2018]; profiles must enforce nonce uniqueness/freshness required by the selected AEAD. The AEAD’s associated-data input carries the  $DS_*$  label together with the version identifier *ver*. For (Wrap, Unwrap), SUDP binds wrapping context through the derivation of  $W$  rather than through an explicit parameter to Wrap. Because  $W_c \leftarrow \text{KDF}(y_c; \eta_c, DS_{\text{wrap}} \parallel cid_c \parallel ver)$  already embeds the credential identifier, salt, and version (§5.2), the interface can be instantiated directly as AES-KW/KWP [Dworkin, 2012, Schaad and Housley, 2002, Housley and Dworkin, 2009] wrapping  $K$  under  $W_c$  without associated data. Profiles that prefer AEAD-as-wrap may additionally authenticate the same domain-separation labels as associated data, yielding the same abstract contract.

**Recipient-protected extraction.** For Phase III.2, the abstract protocol requires a KEM that admits a recipient-specific encapsulation. We instantiate (Encap, Decap) with HPKE [Barnes et al., 2022] rather than an ad-hoc ECIES construction [Abdalla et al., 2001]: HPKE provides a standardized hybrid encryption interface with clear security properties and parameter choices, and it composes cleanly with HKDF-based domain-separation labels.

**Transport.** Pairwise communication runs over TLS 1.3 [Rescorla, 2018]. TLS supplies the authenticated and confidential channel assumed by Phase II on the  $U \rightarrow T$  leg, not a cryptographic layer we extend. Neither TLS session keys nor TLS peer identities enter the protocol’s cryptographic commitments: all protocol-level binding is at the application layer. When  $U$  and  $T$  do not share a

Table 2: SUDP cryptographic profile: abstract primitive interfaces and their standard-component instantiations.

| Abstract primitive                          | Realization                             | Reference                                                            |
|---------------------------------------------|-----------------------------------------|----------------------------------------------------------------------|
| Tamper-resistant module $Aut_c$ ;           | WebAuthn with PRF extension;            | [Hodges et al., 2021, FIDO Alliance, 2025]                           |
| $U$ -verified Sig, PRF                      | ES256/P-256 in this profile             | n/a                                                                  |
| Hash H                                      | SHA-256                                 | [Krawczyk and Eronen, 2010, Barker et al., 2020]                     |
| KDF (extract-and-expand, domain separation) | HKDF-SHA-256                            | [Dworkin, 2007, Nir and Langley, 2018]                               |
| AEAD Enc/Dec                                | AES-GCM or ChaCha20-Poly1305            | [Dworkin, 2012, Schaad and Housley, 2002, Housley and Dworkin, 2009] |
| Key wrap Wrap/Unwrap                        | AES-KW / AES-KWP (or AEAD-as-wrap)      | [Barnes et al., 2022]                                                |
| KEM Encap/Decap                             | HPKE (DHKEM-P256 / HKDF-SHA-256 / AEAD) |                                                                      |

live TLS session, Phase II’s channel assumptions (authenticity and confidentiality) are realized by the cross-device HPKE profile of §7.2. The same profile is required when the deployment topology places a plaintext-inspecting intermediary between  $U$  and  $T$ , for instance an HTTPS relay, L7 proxy, or TLS-terminating load balancer that re-originate the inner connection. Hop-by-hop TLS through such an intermediary does not realize the abstract  $U \rightarrow T$  confidentiality assumption (§5.4), so the cross-device profile must be applied end-to-end across the relay rather than once per hop.

**Sender-constraining view of the grant.** The grant  $G$  is a *sender-constrained* authorization token in the sense of DPoP [Fett et al., 2023]: the signature  $\sigma^*$  binds the challenge  $\beta$  (which commits to  $H(o)$  and the per-grant freshness token  $r$ ) to the per-credential signing key  $sk_{c^*}$ , itself looked up at  $T$  via  $cid_{c^*} \in G$ . Viewed through this lens, II.4’s anti-substitution property is the sender-constraining property instantiated for authenticator-bound authorization, and II.3’s signature verification plus freshness-token consumption is the standard grant-binding check.

Table 2 maps each abstract cryptographic primitive of §5.4 to its concrete realization; transport (TLS or HPKE per §7.2) and the DPoP-style positioning of  $G$  are deployment-level concerns discussed in prose above, not primitive choices.

## 7.2 Cross-Device Channel via HPKE

When  $U$  and  $T$  do not share a live TLS session (the agent runs on hosted infrastructure while  $U$  is on a phone, for example), Phase II’s channel assumptions (authenticity and confidentiality) must be realized by an alternative transport. *This profile is transport-only*: it does not modify the abstract Phase II grant. The signed authorization evidence inside  $G$  is still  $\sigma^* = \text{Sig}_{sk_{c^*}}(\beta)$  over  $\beta = H(\text{DS}_{\text{bind}} \parallel r \parallel H(o))$  (§5.6); the cross-device profile only provides a confidential channel that delivers  $G$  intact from  $U$  to  $T$ .

*Authenticity of  $pk_T$  is required.* Confidentiality of  $W^*$  (carried inside  $G$ ) is security-critical: any party that observes  $W^*$  can directly unwrap  $[K]_{c^*}$  from  $\Sigma$  and decrypt  $M$ . Without an authentic  $pk_T$ , HPKE Base admits an active man-in-the-middle: an adversary substituting its own  $pk_M$  for  $pk_T$  causes  $U$  to seal  $G$  to a key the adversary holds.  $U$  must therefore verify that  $pk_T$  belongs to the intended custodian. A concrete profile may obtain this authenticity by:

- a signature on  $pk_T$  under a long-term  $T$ -instance key whose public counterpart is provisioned to  $U$ ’s trusted client at registration;
- binding  $pk_T$  to an existing authenticated session between  $U$ ’s client and the orchestration service that issued the cross-device link; this requires that the orchestration service be trusted for custodian-key binding, and is therefore unavailable when the deployment treats orchestration as adversarial;
- an out-of-band channel integrity-protected against substitution (e.g., a QR code displayed on a console authenticated to  $U$ );
- a mutually authenticated PAKE or equivalent.

The construction below *assumes* authenticated  $pk_T$ ; without it, the confidentiality argument does not apply.

*Construction.* Given an authentic  $pk_T$ ,  $U$  seals the abstract grant  $G = (o, r, cid_{e^*}, W^*, \sigma^*, opt)$  to  $T$  using HPKE Base mode [Barnes et al., 2022]:

$$(enc, ct_G) \leftarrow \text{HPKE.SealBase}(pk_T, info, \varepsilon, G),$$

with  $info = \text{DS}_{\text{xd}} \parallel r$  binding the freshness token into HPKE’s key schedule (empty AAD).  $U$  transmits  $(enc, ct_G)$  to  $T$  as a CBOR envelope or equivalent, and  $T$  recovers  $G \leftarrow \text{HPKE.OpenBase}(sk_T, enc, info, \varepsilon, ct_G)$  before running Phase II.3 redemption against  $\sigma^*$  and  $\beta$  unchanged.

Under (A2, A5), HPKE Base provides IND-CCA2 sealed delivery to  $pk_T$ ; the *info*-bound  $r$  commits the channel to the freshness token, complementing the protocol-level single-use consumption of  $r$  at Phase II.3. The channel does not supply authorization, which remains  $\sigma^*$  inside  $G$ : a relay that captures  $(enc, ct_G)$  cannot reuse it because  $r$  is consumed at  $T$  on first redemption (step 1), and substitution of  $o$  inside  $G$  fails  $\sigma^*$  verification (step 3). Under the authenticity precondition on  $pk_T$ , the profile does not require a live TLS tunnel between  $U$  and  $T$ ; without that authenticity, the confidentiality of  $W^*$  on the  $U \rightarrow T$  leg is not guaranteed.

### 7.3 Grant Submission Topologies

Phase II’s logical  $U \rightarrow T$  leg admits more than one realization. The authorizer  $U$  (typically a human on a personal device) does not maintain a continuous session with  $T$ , and the party that physically delivers  $G$  varies with deployment. It is convenient to name three sub-actions:  $U$  *authorizes*  $G$  (signs  $\sigma^*$  over  $\beta$ ); a carrier *submits*  $G$  to  $T$ ;  $T$  *redeems*  $G$  (validates and emits  $\rho$ ). Phase II.3 in §5.6 treats submission and redemption together because the core protocol is carrier-agnostic; this section separates them to expose the carrier choice.

**Direct submission.**  $U$  delivers  $G$  to  $T$  over an end-to-end confidential session: TLS 1.3 when  $U$  and  $T$  share a live tunnel, or the cross-device envelope of §7.2 otherwise.  $R$  is not on the submission path and never holds  $G$  in any form.

**Relayed submission.**  $U$  seals  $G$  end-to-end for  $T$  under the envelope of §7.2, then hands the ciphertext to a relay that forwards it to  $T$ . The natural relay in an agentic deployment is  $R$  itself, which lets the agent retry, batch, or schedule submission without an extra coordination service. The relay observes only the opaque HPKE ciphertext sealed to  $pk_T$ ; the inner  $W^*$  stays hidden.

The two topologies share the same security guarantees under the same assumptions:  $\sigma^*$  on  $\beta$  binds the operation independently of the carrier,  $r$  is single-use, and the envelope keeps  $W^*$  end-to-end confidential. They differ in deployment profile. Direct submission keeps the  $U \rightarrow T$  session under  $U$ ’s own control but requires  $T$  to be reachable when  $U$  signs. Relayed submission decouples signing from delivery, letting  $R$  schedule submission and survive transient  $T$  unavailability, at the cost of routing through an entity inside  $R$ ’s trust boundary. Profiles MAY adopt either topology without changing the abstract protocol; the choice is orthogonal to AV, OB, UB, and CRC, which hold from  $\sigma^*$  and  $r$ ’s single-use semantics alone.

## 8 Design Rationale

Several SUDP design decisions are consequential but otherwise scattered across the construction. This section consolidates the architectural-level choices to make their rationale visible together; smaller decisions (domain-separation strings, concrete primitive selections) are discussed inline at the point of construction.

**Per-grant  $U$ -signed authorization committed to  $H(o)$ .** The redeemed grant  $\rho$  is cryptographically bound to a specific operation via  $H(o)$  at the authenticator, not to a stored consent reused across token lifetimes. This lifts anti-replay and anti-substitution from engineering discipline (TTLs, revocation lists, nonce tracking in each tool) to protocol-level guarantees: any captured artifact, within any time window, is redeemable only against the operation it was signed for. SUDP carries the redemption

budget in  $o.\text{valid.multiplicity} \in \{1, \infty\}$ , which  $U$  signs as part of the grant. *Single-redemption* grants (multiplicity=1) match transaction-confirmation ceremonies (PSD2 dynamic linking, banking 2FA): consumed on first use, suited to high-authority actions whose misuse already warrants per-action human oversight. *TTL-bounded multi-redemption* grants (multiplicity= $\infty$ , bounded by expiry) match a multi-entry visa with a fixed validity window:  $U$  explicitly opts into repeated use within a  $U$ -declared rule, suited to low-risk repeat patterns (e.g., inbox reads) where per-action confirmation is operationally untenable. In both modes, the bounds are committed by  $U$ 's per-grant signature, not delegated to an authorization server's policy.

**Authenticator-bound PRF for wrapping material.** Wrapping keys are derived from authenticator-bound PRF outputs ( $y_c = \text{PRF}_c(\eta_c)$  inside  $\text{Aut}_c$ ) rather than from passwords, server-held master keys, or envelope-encryption KMS services. The unlock material therefore lives exclusively in  $U$ 's authenticator hardware: neither the server's persistent state nor its runtime memory can reconstruct a wrapping key without a live  $U$ -verified invocation. This is what distinguishes SUDP's satisfaction of CSB from schemes whose "encrypted at rest" claim collapses when an attacker captures the server together with its unlock material (§4). The cost is a WebAuthn PRF-capable authenticator, or an equivalent authenticator-bound PRF, as a deployment prerequisite. Ordinary passkey availability is broad, but PRF support remains a profile requirement that deployments must check.

**Rewrap policy at rotation.** A SUDP profile inherits the recoverability semantics of its authenticator family: if all enrolled credentials are lost,  $M$  becomes unrecoverable through SUDP. Deployments that need recovery must provision it as an additional enrolled credential (a recovery passkey, a second hardware authenticator, or an enterprise-controlled escrow credential whose use is itself an SUDP-authorized lifecycle operation), and never as a backend master-key bypass, which would surrender CSB. Each enrolled credential therefore holds its own wrapping key  $W_c$ , and every Phase III.3 state-update by one credential must rewrap the new  $K'$  under all peers (§5.8). Three rewrap policies meet this requirement, summarized in Table 3. *In-state peer map* (default) keeps each rewrap single-invocation, at the cost of a transient  $\{W_c\}$  exposure to  $T$  during III.0 and a per-peer forward-secrecy gap until that peer itself rotates. *All-present rotation* removes the exposure but forces every credential online at each rewrap, which is ill-suited to continuous agent workflows. *Append-only chain* supports offline catch-up at unbounded chain storage but is not forward-secret: a captured  $K_j$  walks the chain forward. The default favors single-invocation rewrap for agent workflows that rotate on every write; deployments needing stronger cross-credential isolation may substitute all-present, or adopt append-only as an offline-catchup auxiliary with the FS caveat above.

Table 3: Rewrap policies for multi-credential SUDP rotation.

| Policy                      | Online invocations    | Extra storage              | Peer-key exposure      | FS profile                                |
|-----------------------------|-----------------------|----------------------------|------------------------|-------------------------------------------|
| In-state peer map (default) | 1 (single-invocation) | $O( \text{Creds} )$ in $M$ | transient during III.0 | Lemma 1 per credential, after $c$ rotates |
| All-present rotation        | all credentials       | none                       | none                   | Lemma 1 per credential, independently     |
| Append-only chain           | 1 (single-invocation) | unbounded (chain grows)    | none                   | not FS: old $K_j$ walks forward           |

**Minimality of the binding hash.** The binding hash commits only to freshness and the operation:

$$\beta = \text{H}(\text{DS}_{\text{bind}} \parallel r \parallel \text{H}(o)).$$

The credential identifier is intentionally omitted:  $\text{cid}_{c^*}$  is delivered in  $G$  alongside  $\sigma^*$ , and verification through  $\text{Reg}[\text{cid}_{c^*}] \rightarrow pk_{c^*}$  already enforces credential binding under EUF-CMA; restating it inside  $\beta$  would be redundant. The custodian's identity and the server's TLS certificate hash are likewise omitted: those are bound by the authenticated channel carrying the grant rather than by the application-layer authorization challenge. Re-binding fields already bound elsewhere trades clarity for bloat, and  $\beta$  follows standard key-exchange transcript discipline of carrying only what the surrounding structure does not already commit [Krawczyk, 2005, Rescorla, 2018].

**Per-write rotation by design.** Wrapping-epoch rotation is mandated by the protocol rather than left to operator discipline: every Phase III.3 state-update samples a fresh  $K'$  and a fresh next-salt  $\eta_{c^*}^{\text{next}}$  (§5.8). Coupling rotation to every write removes the need for a separate rotation ceremony and delivers wrapping-epoch forward secrecy unconditionally under key-erasure assumptions (Proposition 4, Lemma 1). Authority-level rotation (rotating the underlying secret at  $E$ ) is kept separate, runs on a deployment-defined cadence, and depends on  $E$ -side revocation support (Proposition 5); the

separation lets wrap-side FS hold independently of  $E$ 's rotation API. The per-write cost is one CSPRNG draw, one key derivation, and  $|\text{Creds}|$  rewraps, which is marginal for agent workflows whose write volumes are typically low per user.

## 9 Security Analysis

We state SUDP's security as a single main theorem (SUDP solves the Agent Secret Use problem), parameterized by explicit assumptions and three named interface preconditions (§9.2). The proof decomposes into five propositions (§9.4), and §9.5 maps the result back to the seven taxonomy axes. Proofs are sketched; a mechanized analysis in a tool such as Tamarin [Meier et al., 2013] or ProVerif [Blanchet, 2001] is left to future work.

### 9.1 Security Statement

Informally, SUDP solves the Agent Secret Use problem against any adversary controlling  $R$  and all requester-visible or non-trusted routing infrastructure: every Phase III execution is covered by a fresh,  $U$ -rooted, operation-bound, use-bounded grant, and  $R$ 's view across all sessions is restricted to public proposal material plus the authorized result form  $\text{Release}(o)$  returned by  $T$  (§5.7), which by construction carries no authority-bearing material. Residual limitations outside this claim are catalogued in §9.6.

### 9.2 Assumptions and Interface Preconditions

#### Assumptions.

- (A1) EUF-CMA security of the signature scheme  $\text{Sig}$ .
- (A2) IND-CCA security of the AEAD scheme.
- (A3) PRF security of the authenticator  $\text{Aut}_c$  (WebAuthn PRF extension).
- (A4) Collision resistance of  $H$  combined with deterministic, injective canonical serialization of  $o$ .
- (A5) IND-CCA2 security of the KEM (HPKE).
- (A6) Authenticated and confidential channel  $U \rightarrow T$  for the leg carrying  $W^*$  (TLS 1.3, or the cross-device profile of §7.2 under its authenticated-key precondition).
- (A7) Single-use freshness-state integrity at  $T$ :  $r$  is consumed exactly once, and  $S$  is not adversarially modified between issuance and redemption (no  $T$  memory compromise during this interval).
- (A8) (Conditional, used only by Proposition 5.)  $E$  supports rotation and revocation of authority-bearing secrets on the deployment's rotation cadence.
- (A9) Runtime hygiene at  $T$ : fresh draws are CSPRNG-quality (output computationally indistinguishable from uniform and independent of prior draws), and retired epoch keys ( $K$ ,  $W^*$ , peer wrapping keys) are erased after every wrap-epoch advance (§5.8).

#### Interface preconditions.

- (P1) *Canonical Operation Completeness.* Every authority-relevant field of the action exercised at  $E$  is represented in  $o.\text{act.scope}$ ,  $o.\text{bind}$ , or  $o.\text{valid}$  (§5.3).
- (P2) *Trusted Rendering Faithfulness.*  $\text{Render}(o)$  at  $U$  is deterministic, complete with respect to the fields in  $o$ , and produced inside  $U$ 's trusted client (§5.3).
- (P3) *Execution Determinism.* The native call performed at  $E$  by  $T$  is a deterministic function of accepted  $o$ , the protected material released inside  $T$ , and any  $o$ -committed payload;  $R$  cannot supply authority-relevant fields after redemption (§5.3).

### 9.3 Main Theorem

**Theorem 1 (SUDP solves the Agent Secret Use problem).** *Under (A1–A7) and interface preconditions (P1–P3), SUDP enforces the four core security properties of §3.3: AV, OB, UB, and CRC. By the criterion of §3.3, SUDP therefore solves the Agent Secret Use problem (Definition 3) against any adversary controlling  $R$  and all requester-visible or non-trusted routing infrastructure, provided*

the  $U \rightarrow T$  grant channel satisfies the authenticity and confidentiality assumptions stated in (A6). Concretely:

1. Every Phase III execution admitted by  $T$  is covered by a fresh,  $U$ -rooted, operation-bound, use-bounded grant (AV, OB, UB);
2.  $R$ 's view across all sessions consists of public proposal material and  $\text{Release}(o)$ , and excludes any reusable authority or transferable artifact (CRC).

SUDP additionally satisfies the robustness extensions CSB and RFS-wrap under (A2, A3, A9); authority-level forward secrecy (RFS-auth) holds under the further assumption (A8) via Proposition 5. Runtime-memory confidentiality of  $T$  during the unwrap-use-zeroize window (CMC) is out of scope; it is the natural composition target for TEE-backed custodians.

The proof decomposes into five propositions that together discharge Theorem 1: Proposition 1 discharges item 1 (AV, OB, UB); Proposition 2 discharges item 2 (CRC) and additionally establishes CSB; Proposition 3 establishes the ASU-preserving export case as a refinement of item 2; Proposition 4 establishes RFS-wrap; Proposition 5 establishes the conditional RFS-auth.

## 9.4 Supporting Propositions

**Proposition 1 (Authorization Integrity:  $\text{AV} \wedge \text{OB} \wedge \text{UB}$ ).** *Under (A1, A4, A7) and (P1–P3), every Phase III execution that  $T$  admits carries a signature  $\sigma^*$  verifiable as a  $U$ -rooted authenticator assertion under  $\text{Reg}[cid_{c^*}]$  over  $\beta = \text{H}(\text{DS}_{\text{bind}} \parallel r \parallel \text{H}(o))$  (AV); no valid grant admits any  $o' \not\equiv_E o$  (OB); and every Phase III execution stays within the bounds  $o.\text{valid} = (\text{expiry}, \text{multiplicity})$  signed by  $\sigma^*$  (UB).*

*Proof sketch.* Phase II.3 (§5.6) verifies  $\text{Vrfy}_{\text{Reg}[cid_{c^*}]}(\beta', \sigma^*) = 1$  and consumes  $r$  from  $S$  before any Phase III execution. AV: under (A1), only the holder of  $sk_{c^*}$  (i.e.,  $U$  via  $\text{Aut}_{c^*}$ ) can produce a  $\sigma^*$  accepted by this check;  $\text{Reg}[cid_{c^*}]$  is public, so any auditor with the transcript can re-run the verification. OB: under (A4) and (P1), if any  $o' \not\equiv_E o$  is presented, the recomputed  $\beta'$  diverges from the signed  $\beta$  and verification fails; if the acting credential identifier in  $G$  is altered,  $\text{Vrfy}$  is invoked under a non-matching  $pk$  and rejects  $\sigma^*$  under (A1). (P2) ensures  $U$  authorized the canonical  $o$  that  $T$  validates, so  $U$  cannot be tricked into authorizing one operation while  $T$  executes another; (P3) prevents  $R$  from injecting authority-relevant fields after redemption. UB:  $\sigma^*$  commits to  $\text{H}(o)$ , and  $o.\text{valid}$  is part of  $o$ ;  $T$  enforces  $o.\text{valid}$  at every Phase III invocation, so execution stays within the bounds  $U$  signed.  $r$ 's single-use consumption at II.3 (A7) prevents manufacturing additional grants from a captured  $(o, r, \sigma^*)$  tuple; forging a fresh  $\sigma^*$  on a new  $r$  requires  $sk_{c^*}$  (A1).  $\square$

**Proposition 2 (Secret Confidentiality:  $\text{CRC} \wedge \text{CSB}$ ).** *In the trust model of §3.1: under (A2, A3, A6),  $R$ 's view across all sessions is bounded by public proposal material and the authorized result form  $\text{Release}(o)$  of accepted operations (§5.7); it contains no value from which  $s$ ,  $K$ ,  $W_c$ , or  $s_o$  can be recovered (CRC). Independently, under (A2, A3), an adversary obtaining the full persistent state  $\Sigma$  without a live  $U$ -verified  $\text{Aut}_c$  invocation cannot recover  $M$  (CSB).*

*Proof sketch.* CRC:  $\Sigma$  exposes only public  $\{C, \{[K]_c\}, \{\eta_c\}, \text{Reg}\}$ ;  $K$  and  $\{W_c\}$  materialize only inside  $T$  under a  $U$ -verified invocation.  $R$  observes the public hand-off  $(o, r, cid_c, \eta_c)$  and  $\text{Release}(o)$ ; under (A6), the  $U \rightarrow T$  leg of  $G$  is confidential, so  $R$  never observes  $W^*$ .  $\sigma^*$  is public authorization evidence and its visibility to  $R$  is immaterial: redemption requires the matching unconsumed  $r$  and the secret  $W^*$ , neither of which  $R$  obtains. CSB: recovering  $M$  from  $\Sigma$  requires  $K$ , hence some  $W_c$ , hence  $y_c = \text{PRF}_c(\eta_c)$ ; under (A3), no efficient adversary without a fresh  $\text{Aut}_c$  invocation can produce  $y_c$ ; under (A2) and key-wrap security, the AEAD/key-wrap ciphertexts hide the underlying plaintext from any party without the key.  $\square$

**Proposition 3 (Recipient-Protected Export).** *Given an accepted operation  $o$  satisfying Proposition 1, and under (A5, A2), for any ASU-preserving Phase III.2 execution (a well-formed  $o$  whose  $\text{bind.recipient}$  identifies a principal outside  $R$ 's trust boundary), the delivery  $\pi$  reveals nothing about  $s_o$  to any party other than the holder of  $sk_{\text{rcp}}$ .*

*Proof sketch.*  $\pi = (ct_d, \delta)$  couples an IND-CCA2 KEM ciphertext under  $pk_{rcp}$  with an AEAD ciphertext under  $k_d$ , derived from the KEM output and committing to  $H(o)$ . Without  $sk_{rcp}$ ,  $\pi$  reveals nothing about  $s_o$  under (A5) and (A2). Recipient substitution fails because  $pk_{rcp}$  is in  $o.\text{bind}$ , hence in  $\beta$ ; changing it invalidates  $\sigma^*$  (Proposition 1). A grant whose recipient names  $R$  itself is a deliberate ownership transfer outside ASU non-disclosure (§5.7).  $\square$

**Proposition 4 (Wrapping-Epoch Key Isolation).** *Under (A2, A3, A9), a Phase III.3 update yields wrapping-epoch key isolation: the post-rotation keys  $(K', W_{\text{new}}^*)$  are computationally independent of the pre-rotation keys  $(K, W_{c^*})$ ; compromise of either epoch’s keys does not enable deriving the other epoch’s keys or decrypting retained ciphertexts encrypted under the other epoch’s key.*

*Proof sketch.*  $K'$  is sampled fresh and independent of  $K$  (CSPRNG); under (A2), ciphertexts under  $K'$  are indistinguishable from random given any number of  $K$ -decryptions, and symmetrically for ciphertexts under  $K$  given  $K'$ -decryptions.  $W_{\text{new}}^*$  is derived through the chain  $\text{PRF}_{c^*} \rightarrow \text{KDF} \rightarrow \text{KDF}$  rooted in a fresh next-salt  $\eta_{c^*}^{\text{next}}$ ; per Lemma 1 (per-credential FS), the chain breaks the derivation link in both directions. The claim applies to credentials that have actually rotated; dormant-credential and peer-map limitations are L3 and L4 below.  $\square$

**Plaintext caveat.** Wrapping-epoch key isolation does not by itself protect plaintext values that are intentionally carried forward across rotations. The rewrap and pure-rotate sub-cases of Phase III.3 explicitly preserve  $M' = M$  (§5.8); under those operations, the plaintext present in current state equals the plaintext recoverable from the previous state, so an adversary who decrypts either epoch’s ciphertext recovers the same  $M$ . Plaintext-level forward secrecy of authority-bearing material requires either (i) a Phase III.3 update that retires the prior secret in  $M$ , or (ii) composition with  $E$ -side authority rotation (Proposition 5), which decouples plaintext continuity from authority continuity.

**Proposition 5 (Authority-Level Forward Secrecy under E-side rotation).** *Under (A8) in addition to the assumptions of Proposition 4, compromise of one epoch’s authority-bearing secret at  $E$  confers no authority over operations executed in any other epoch.*

*Proof sketch.* By (A8), each epoch executes against an authority-bearing secret issued for that epoch and revoked at the next rotation. Authority over a retired epoch therefore requires a secret  $E$  no longer accepts; the same argument runs in reverse for a current-epoch operation under a retired secret. Combined with Proposition 4’s wrapping-side forward secrecy, authority is bounded to its issuing epoch. Without (A8), the claim degrades to wrapping-epoch isolation only.  $\square$

## 9.5 Taxonomy-Axis Coverage

Table 4: Taxonomy-axis coverage. Theorem 1 provides the headline guarantee; each axis is established by one of the supporting propositions, with explicit dependencies on assumptions (A1–A9, §9.2) and interface preconditions (P1–P3, §9.2). CMC is out of scope at the protocol level (L1, §9.6) and is closed by composition with a hardware-rooted runtime.

| Axis                             | Established by    | Key assumptions                      |
|----------------------------------|-------------------|--------------------------------------|
| AV (Authorization Verifiability) | Prop. 1           | A1, A4, P1–P2                        |
| OB (Operation Binding)           | Prop. 1           | A1, A4, P1–P3                        |
| UB (Use Boundedness)             | Prop. 1           | A1, A7                               |
| CRC (Requester Compromise)       | Prop. 2, Prop. 3  | A2, A3, A5, A6                       |
| CSB (Storage Breach)             | Prop. 2           | A2, A3                               |
| RFS (Rotation Forward Secrecy)   | Prop. 4, Prop. 5  | A2, A3, A9 (+A8 for authority-level) |
| CMC (Runtime Memory)             | out of scope (L1) | n/a                                  |

## 9.6 Residual Limitations

Five residual limitations (L1–L5) are explicit in the design and should be disclosed to any deployment.

(L1) *Runtime compromise during consumption.* A fully compromised  $T$  runtime during legitimate consumption exfiltrates plaintext by construction: III.0 materializes  $K$  and  $M$  inside  $T$ ’s memory,



and no protocol-level mechanism prevents code executing inside  $T$  from reading them. Composition with confidential-computing runtimes is the natural remediation. As a defense-in-depth note (not a protocol-level guarantee), a deployment can narrow exposure to the unwrap-use-zeroize critical section by zeroing  $K$ ,  $W^*$ , and any derived plaintext immediately after use, disabling swap and core dumps for  $T$ , and gating against memory inspection by co-tenant processes; closing CMC requires hardware-rooted memory protection, typically via TEE composition.

(L2) *Authenticator compromise.* A fully compromised authenticator  $Aut_{c^*}$  reduces Phase II.2 to adversary-controlled authorization; authenticator security (A3) is an assumption, not a protocol-level property.

(L3) *Dormant-credential forward-secrecy gap.* Lemma 1’s forward secrecy is per-credential and activates only after a credential rotates. A credential that is registered but never participates in a state-update retains its initial salt indefinitely; historical PRF capture, if it ever occurred, remains useful against its wrapping entry until the credential is exercised. This is intrinsic to any-of- $N$  authorization without a global ceremony; deployments should track per-credential last-rotation timestamps and drive scheduled rewrap under the in-state peer-map construction (§5.8).

(L4) *Peer-map cross-credential exposure.* Under the in-state peer-map construction, Phase III.0 by any credential  $c$  transiently exposes the current  $\{W_c\}_{c \in \text{Creds}}$  to  $T$ . Compromise during one credential’s invocation yields the current wrapping keys of all peers; a captured  $W_{c'}$  continues to unwrap subsequent rewraps until  $c'$  itself rotates. Deployments requiring stronger cross-credential isolation may substitute the all-present alternative of §5.8.

(L5) *Authorization is bounded by  $U$ ’s judgment.* SUDP enforces the operations  $U$  signs, not the operations  $U$  should have signed: a dangerous-but-correctly-rendered descriptor approved by  $U$  is executed. The semantic content of a conformant  $\text{Release}(o)$  (e.g., an authorized downstream API response that is itself sensitive) is similarly a deployment-policy concern outside protocol scope.

## 10 Conclusion

Agents that act autonomously on a user’s behalf must be able to use the user’s secrets without ever gaining reusable authority over them. We frame this as the *Agent Secret Use* problem and decompose it along seven security properties (AV, OB, UB for authorization integrity; CRC, CSB, CMC, RFS for secret confidentiality), which together provide a common vocabulary for future work in this area.

We also give a concrete protocol, SUDP, that satisfies six of the seven properties under standard cryptographic assumptions; the seventh, CMC, is closed by composition with a hardware-rooted runtime. The accompanying agentic-system interface specifies where SUDP applies in an LLM-driven agent: primarily at the tool-call layer whenever a call would exercise user-enrolled authority-bearing material, with the LLM-API call entering the same path only in deployments that treat the model-provider key as user authority.

Three directions stand out as natural next steps: formal verification of SUDP under active adversaries; empirical evaluation of approval cost under realistic agent workloads; and the open design question of reconciling agent autonomy with the granularity of authorizer-signed grants, whether by data-driven optimization of grant bounds or composition with policy systems.

## References

- Michel Abdalla, Mihir Bellare, and Phillip Rogaway. The oracle Diffie-Hellman assumptions and an analysis of DHIES. In *Topics in Cryptology—CT-RSA 2001*, pages 143–158. Springer, 2001.
- Amazon Web Services. AWS Secrets Manager: Rotate, manage, and retrieve secrets. In *AWS Documentation*, 2024. URL <https://docs.aws.amazon.com/secretsmanager/>.
- Anthropic. The Claude 3 model family: Opus, sonnet, haiku. Anthropic Model Card, 2024a. URL <https://www.anthropic.com/claude>.
- Anthropic. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>, 2024b.

- Anthropic. Claude Agent SDK: Secure deployment patterns. <https://code.claude.com/docs/en/agent-sdk/secure-deployment>, 2026.
- Anthropic and MCP Working Group. Model context protocol authorization specification. Technical report, Model Context Protocol, 2025. URL <https://modelcontextprotocol.io/specific-ation/2025-06-18/basic/authorization>.
- Elaine Barker, Lily Chen, and Richard Davis. Recommendation for key-derivation methods in key-establishment schemes. Technical Report NIST Special Publication 800-56C Rev. 2, National Institute of Standards and Technology, 2020.
- Richard Barnes, Karthikeyan Bhargavan, Benjamin Lipp, and Christopher A. Wood. Hybrid public key encryption. *RFC 9180*, 2022. doi: 10.17487/RFC9180. URL <https://www.rfc-editor.org/rfc/rfc9180>.
- Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentzner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *21st Annual Network and Distributed System Security Symposium (NDSS)*, 2014.
- Bruno Blanchet. An efficient cryptographic protocol verifier based on Prolog rules. In *14th IEEE Computer Security Foundations Workshop (CSFW)*, pages 82–96, 2001.
- Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. *arXiv preprint arXiv:2402.06363*, 2024a. URL <https://arxiv.org/abs/2402.06363>.
- Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending against prompt injection with preference optimization. *arXiv preprint arXiv:2410.05451*, 2024b. URL <https://arxiv.org/abs/2410.05451>.
- Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. Credential leakage in LLM agent skills: A large-scale empirical study. *arXiv preprint arXiv:2604.03070*, 2026. URL <https://arxiv.org/abs/2604.03070>.
- Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. LlamaFirewall: An open source guardrail system for building secure AI agents. *arXiv preprint arXiv:2505.03574*, 2025. URL <https://arxiv.org/abs/2505.03574>.
- Victor Costan and Srinivas Devadas. Intel SGX explained. IACR Cryptology ePrint Archive 2016/086, 2016.
- CrewAI. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>, 2024.
- Asiri Dalugoda. HDP: A lightweight cryptographic protocol for human delegation provenance in agentic AI systems. *arXiv preprint arXiv:2604.04522*, 2026. URL <https://arxiv.org/abs/2604.04522>.
- Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *The Thirty-eighth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=m1YYAQj03w>.
- Edoardo DeBenedetti, Iliia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025. URL <https://arxiv.org/abs/2503.18813>.
- Xinhao Deng, Yixiang Zhang, Jiaqing Wu, Jiaqi Bai, Sibao Yi, Zhuoheng Zou, Yue Xiao, Rennai Qiu, Jianan Ma, Jialuo Chen, Xiaohu Du, Xiaofang Yang, Shiwen Cui, Changhua Meng, Weiqiang Wang, Jiaying Song, Ke Xu, and Qi Li. Taming OpenClaw: Security analysis and mitigation of autonomous LLM agent threats. *arXiv preprint arXiv:2603.11619*, 2026. URL <https://arxiv.org/abs/2603.11619>.

- Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. Technical Report 3, Communications of the ACM, 1966.
- Morris J. Dworkin. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. *NIST Special Publication 800-38D*, 2007.
- Morris J. Dworkin. Recommendation for block cipher modes of operation: Methods for key wrapping. *NIST Special Publication 800-38F*, 2012.
- Faouzi El Yagoubi, Godwin Badu-Marfo, and Ranwa Al Mallah. AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems. *arXiv preprint arXiv:2602.11510*, 2026. URL <https://arxiv.org/abs/2602.11510>.
- Daniel Fett, Brian Campbell, John Bradley, Torsten Lodderstedt, Michael B. Jones, and David Waite. OAuth 2.0 demonstrating proof of possession (DPoP). *RFC 9449*, 2023. URL <https://www.rfc-editor.org/rfc/rfc9449>.
- FIDO Alliance. CTAP2 extensions: PRF and large blob storage. In *Client to Authenticator Protocol (CTAP) v2.2*, 2025. URL <https://fidoalliance.org/specs/fido-v2.2-ps-20250228/fido-client-to-authenticator-protocol-v2.2-ps-20250228.html>.
- Abhishek Goswami. Agentic JWT: A secure delegation protocol for autonomous AI agents. *arXiv preprint arXiv:2509.13597*, 2025. URL <https://arxiv.org/abs/2509.13597>.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *ACM AISec Workshop*, 2023.
- Dick Hardt. The OAuth 2.0 authorization framework. *RFC 6749*, 2012.
- Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988. doi: 10.1145/54289.871709.
- HashiCorp. HashiCorp Vault: Secrets management and data protection. In *HashiCorp Documentation*, 2024a. URL <https://www.vaultproject.io/docs>.
- HashiCorp. HashiCorp vault: AI agent identity pattern. <https://developer.hashicorp.com/validated-patterns/vault/ai-agent-identity-with-hashicorp-vault>, 2024b.
- Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kıcıman. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720*, 2024. URL <https://arxiv.org/abs/2403.14720>.
- Jeff Hodges, J.C. Jones, Michael B. Jones, Akshay Kumar, and Emil Lundberg. Web Authentication: An API for accessing Public Key Credentials level 2. W3C Recommendation, 2021. URL <https://www.w3.org/TR/webauthn-2/>.
- Russ Housley and Morris Dworkin. Advanced encryption standard (AES) key wrap with padding algorithm. Technical Report RFC 5649, Internet Engineering Task Force, 2009. URL <https://www.rfc-editor.org/rfc/rfc5649>.
- Michael Jones and Dick Hardt. The OAuth 2.0 authorization framework: Bearer token usage. Technical Report RFC 6750, Internet Engineering Task Force, 2012. URL <https://www.rfc-editor.org/rfc/rfc6750>.
- Juhee Kim, Xiaoyuan Liu, Zhun Wang, Shi Qiu, Bo Li, Wenbo Guo, and Dawn Song. The attack and defense landscape of agentic AI: A comprehensive survey. *USENIX Security Symposium*, 2026. URL <https://arxiv.org/abs/2603.11088>.
- Hugo Krawczyk. HMQV: A high-performance secure Diffie-Hellman protocol. In *Advances in Cryptology – CRYPTO 2005*, pages 546–566. Springer, 2005.
- Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). *RFC 5869*, 2010.

- LangChain. LangChain: Building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>, 2024.
- Ninghui Li, Kaiyuan Zhang, Kyle Polley, and Jerry Ma. Security considerations for artificial intelligence agents. *arXiv preprint arXiv:2603.12230*, 2026. URL <https://arxiv.org/abs/2603.12230>.
- Junda Lin, Zhaomeng Zhou, Zhi Zheng, Shuochen Liu, Tong Xu, Yong Chen, and Enhong Chen. VIGIL: Defending LLM agents against tool stream injection via verify-before-commit. *arXiv preprint arXiv:2601.05755*, 2026. URL <https://arxiv.org/abs/2601.05755>.
- Songyang Liu, Chaozhuo Li, Chenxu Wang, Jinyu Hou, Zejian Chen, Litian Zhang, Zheng Liu, Qiwei Ye, Yiming Hei, Xi Zhang, and Zhongyuan Wang. ClawKeeper: Comprehensive safety protection for OpenClaw agents through skills, plugins, and watchers. *arXiv preprint arXiv:2603.24414*, 2026. URL <https://arxiv.org/abs/2603.24414>.
- Torsten Lodderstedt, Justin Richer, and Brian Campbell. OAuth 2.0 rich authorization requests. <https://www.rfc-editor.org/rfc/rfc9396>, 2023. RFC 9396.
- Hanjun Luo, Shenyu Dai, Chiming Ni, Xinfeng Li, Guibin Zhang, Kun Wang, Tongliang Liu, and Hanan Salam. AgentAuditor: Human-level safety and security evaluation for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2506.00641>.
- Manus Team. Manus: What we saw in the past three months and what we see in the future. <https://manus.im/blog/what-we-saw-in-the-past-three-months-and-what-we-see-in-the-future>, 2025.
- John McCumber. Information systems security: A comprehensive model. In *Proceedings of the 14th National Computer Security Conference*, pages 328–337, 1991.
- Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *Computer Aided Verification (CAV)*, pages 696–701, 2013.
- Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. Augmented language models: a survey. *Transactions on Machine Learning Research*, 2023.
- Mark Samuel Miller. Robust composition: Towards a unified approach to access control and concurrency control. In *Ph.D. Dissertation, Johns Hopkins University*, 2006.
- Subramanya Nagabhushanaradhya. OpenID Connect for Agents (OIDC-A) 1.0: A standard extension for LLM-based agent identity and authorization. *arXiv preprint arXiv:2509.25974*, 2025. URL <https://arxiv.org/abs/2509.25974>. Individual proposal; not an official OpenID Foundation specification.
- Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, Abhradeep Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections. *arXiv preprint arXiv:2510.09023*, 2025. URL <https://arxiv.org/abs/2510.09023>.
- NEAR AI. IronClaw: An open-source secure runtime for AI agents. <https://github.com/nearai/ironclaw>, 2026.
- Yoav Nir and Adam Langley. ChaCha20 and Poly1305 for IETF protocols. Technical Report RFC 8439, RFC Editor, 2018. URL <https://www.rfc-editor.org/rfc/rfc8439>.
- OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- OpenAI. OpenAI Agents SDK. <https://github.com/openai/openai-agents-python>, 2025a.

- OpenAI. Introducing Operator. <https://openai.com/index/introducing-operator/>, 2025b.
- OWASP Foundation. OWASP Top 10 for large language model applications. *OWASP Project*, 2024. URL <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- Yuxuan Qiao, Dongqin Liu, Hongchang Yang, Wei Zhou, and Songlin Hu. Agent tools orchestration leaks more: Dataset, benchmark, and mitigation. *arXiv preprint arXiv:2512.16310*, 2025. doi: 10.48550/arXiv.2512.16310. URL <https://arxiv.org/abs/2512.16310>.
- Eric Rescorla. The transport layer security (TLS) protocol version 1.3. *RFC 8446*, 2018.
- Justin (Ed.) Richer. G NAP: Grant negotiation and authorization protocol. <https://www.rfc-editor.org/rfc/rfc9635>, 2024. RFC 9635.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023. doi: 10.48550/arXiv.2309.15817. URL <https://arxiv.org/abs/2309.15817>.
- Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. Trusted execution environment: what it is, and what it is not. In *IEEE Trustcom/BigDataSE/ISPA*, pages 57–64. IEEE, 2015.
- Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- Jim Schaad and Russ Housley. Advanced encryption standard (AES) key wrap algorithm. *RFC 3394*, 2002.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.
- Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control. *arXiv preprint arXiv:2504.11703*, 2025. URL <https://arxiv.org/abs/2504.11703>.
- Significant Gravitas. AutoGPT. <https://github.com/Significant-Gravitas/AutoGPT>, 2023.
- Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized AI agents. *arXiv preprint arXiv:2501.09674*, 2025. URL <https://arxiv.org/abs/2501.09674>.
- Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. SAGA: A security architecture for governing AI agentic systems. *arXiv preprint arXiv:2504.21034*, 2025. URL <https://arxiv.org/abs/2504.21034>.
- Lillian Tsai and Eugene Bagdasarian. Contextual agent security: A policy for every purpose. In *Workshop on Hot Topics in Operating Systems (HotOS)*, 2025. doi: 10.1145/3713082.3730378.
- Sanidhya Vijayvargiya, Aditya Bharat Soni, Xuhui Zhou, Zora Zhiruo Wang, Nouha Dziri, Graham Neubig, and Maarten Sap. OpenAgentSafety: A comprehensive framework for evaluating real-world AI agent safety. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2507.06134>.
- W3C Web Payments Working Group. Secure payment confirmation. <https://www.w3.org/TR/secure-payment-confirmation/>, 2026. W3C Candidate Recommendation Draft.
- Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. *arXiv preprint arXiv:2503.18666*, 2025. URL <https://arxiv.org/abs/2503.18666>.

- Sam Warren. Aegis: A credential isolation proxy for AI agents. <https://github.com/getaegis/aegis>, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. <https://arxiv.org/abs/2308.08155>, 2023.
- Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Network and Distributed System Security Symposium (NDSS)*, 2025. doi: 10.14722/ndss.2025.241131. URL <https://arxiv.org/abs/2403.04960>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023.
- Zonghao Ying, Xiao Yang, Siyang Wu, Yumeng Song, Yang Qu, Hainan Li, Tianlin Li, Jiakai Wang, Aishan Liu, and Xianglong Liu. Uncovering security threats and architecting defenses in autonomous agents: A case study of OpenClaw. *arXiv preprint arXiv:2603.12644*, 2026. URL <https://arxiv.org/abs/2603.12644>.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.findings-acl.624/>.
- Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2410.02644>.
- Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-safetybench: Evaluating the safety of LLM agents. *arXiv preprint arXiv:2412.14470*, 2024. doi: 10.48550/arXiv.2412.14470. URL <https://arxiv.org/abs/2412.14470>.
- Xuhui Zhou, Hyunwoo Kim, Faeze Brahman, Liwei Jiang, Hao Zhu, Ximing Lu, Frank F. Xu, Bill Yuchen Lin, Yejin Choi, Niloofar Miresghallah, Ronan Le Bras, and Maarten Sap. Haicosystem: An ecosystem for sandboxing safety risks in interactive AI agents. In *Conference on Language Modeling (COLM)*, 2025. URL <https://openreview.net/forum?id=KI1WQ6rLiy>.